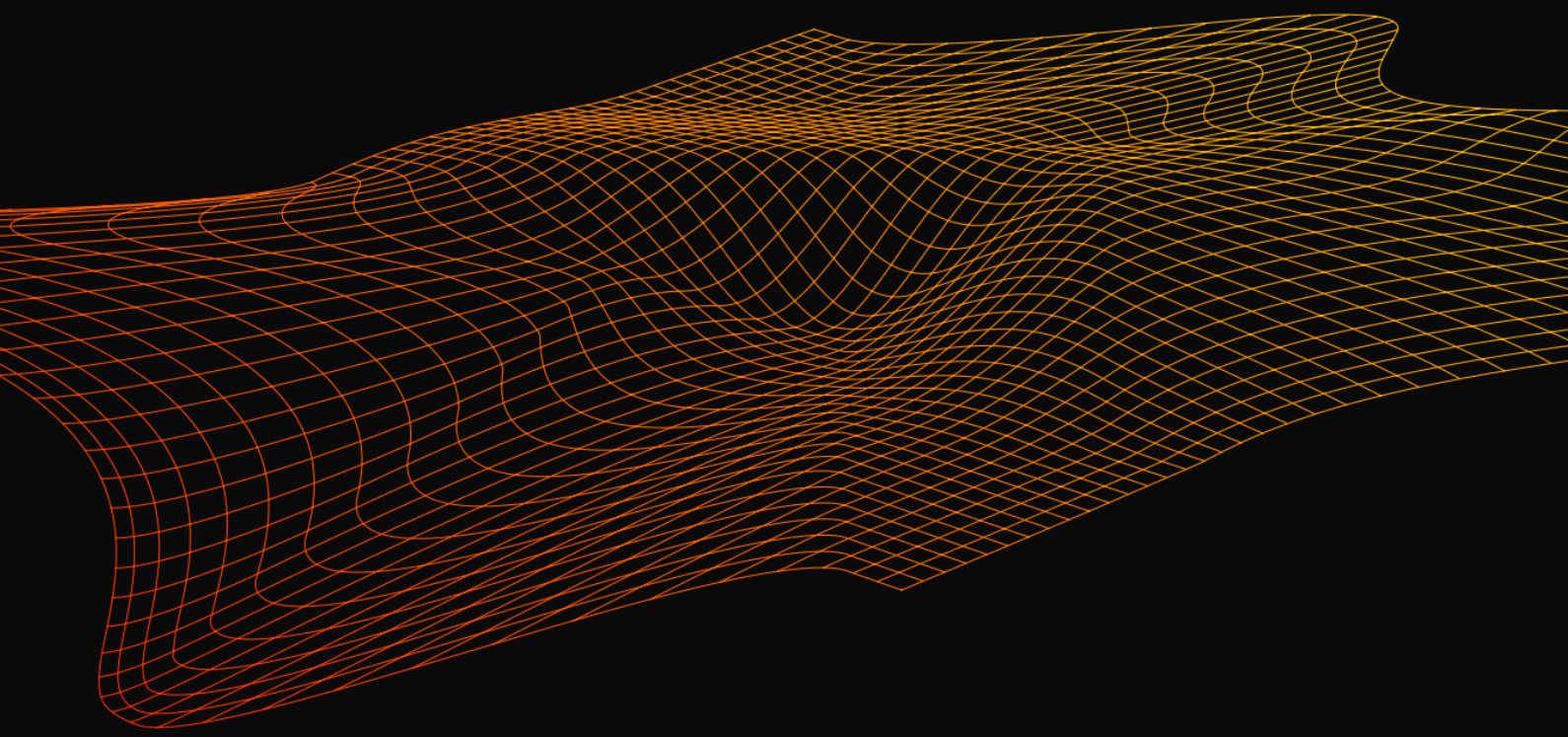
TAREA UNIDAD 1

Introducción a Hadoop

Bases de Datos Aplicadas (BDA)



Inteligencia Artificial y Big Data

Ana Isabel Navarro Jiménez

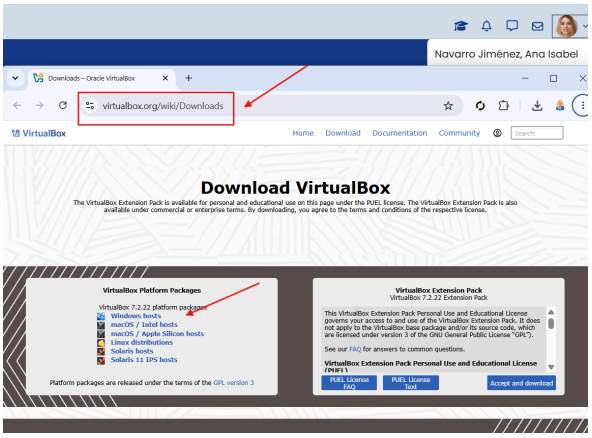
ÍNDICE

1. Creación de la máquina virtual....3
2. Configuración del nombre del equipo y preparación del sistema....5
3. Creación del usuario Hadoop....9
4. Configuración de SSH....9
5. Instalación de Apache Hadoop....11
6. Configuración pseudodistribuida....13
7. Formateo y puesta en marcha de HDFS y YARN....15
8. Interfaces web de Hadoop....16
9. SIstemas de almacenamiento....19
10. Creación de la estructura HDFS....19
11. Preparación del conjunto de datos....20
12. Ingestión de datos en HDFS....22
13. Análisis de formatos de almacenamiento....23
14. Comprobación del almacenamiento físico en HDFS....24
15. Procesamiento de los datos....26
16. Hadoop Streaming con Python....27
17. Análisis de la ejecución....29
18. Presentación e interpretación de resultados....30
    1. Objetivo....30
    2. Indicadores de rendimiento....30
    3. Tabla resumen....30
    4. Gráficos....30
    5. Conclusiones....30
19. ¿Qué ventajas aporta HDFS frente al almacenamiento tradicional?....31
20. ¿Qué papel desempeña la replicación en Apache Hadoop?....31
21. ¿Qué ocurriría si un DataNode dejará de funcionar en un clúster real?....31
22. ¿Qué ventajas aporta separar almacenamiento y procesamiento?....32
23. ¿Qué problemas o limitaciones presenta utilizar Apache Hadoop con un único nodo?....32
24. ¿Qué cambios serían necesarios para convertir esta instalación en un clúster de tres nodos? 32
25. ¿Qué formato de datos considera más adecuado para el problema propuesto? Justifica la respuesta....32
26. ¿Qué utilidad tiene Hadoop Streaming?....33
27. ¿Qué función desempeña YARN en el procesamiento de los datos?....33
28. A partir de los resultados obtenidos, ¿Qué conclusiones principales comunicarías al cliente final?....33

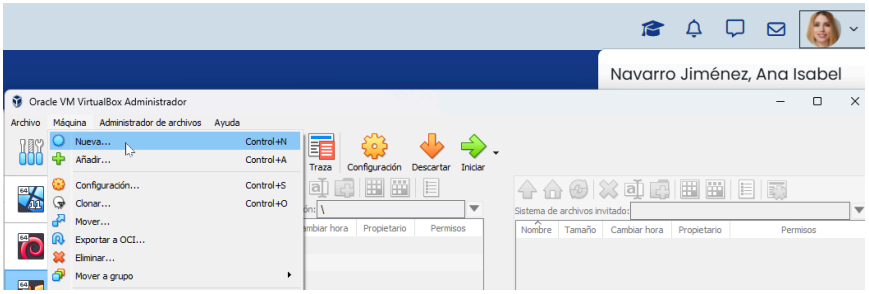
1. Creación de la máquina virtual

Crea en Oracle VirtualBox una máquina virtual con Ubuntu Server 26.04 LTS, 2 procesadores virtuales, 8 GB de RAM, 100 GB de almacenamiento y una configuración de red que permita acceso a Internet y a las interfaces web de Hadoop desde el equipo anfitrión.

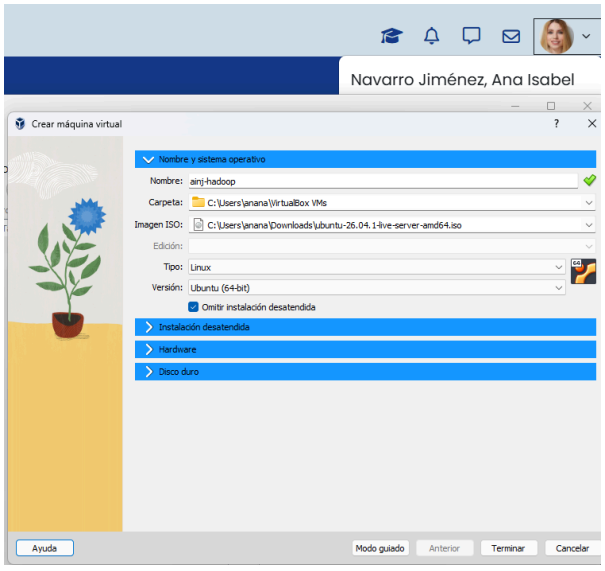
Para la creación de la máquina virtual se debe comenzar instalando el programa que nos permita realizar dicha creación. En mi caso, ya tenía la instalación de VirtualBox realizada por lo que saltaré los pasos de instalación, pero solo se debe seguir los pasos que indica la web y descargamos la iso de la versión 26



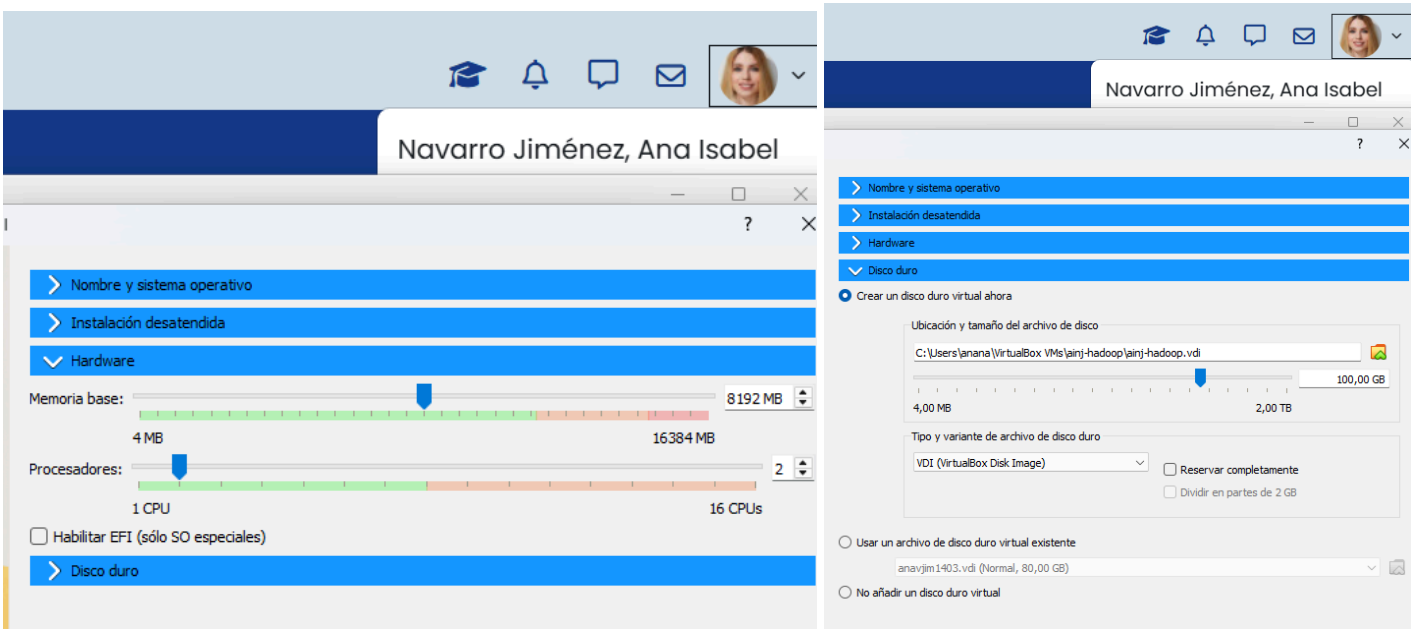
Una vez tenemos VirtualBox vamos a hacer la creación de la máquina: ***Máquina > Nueva***



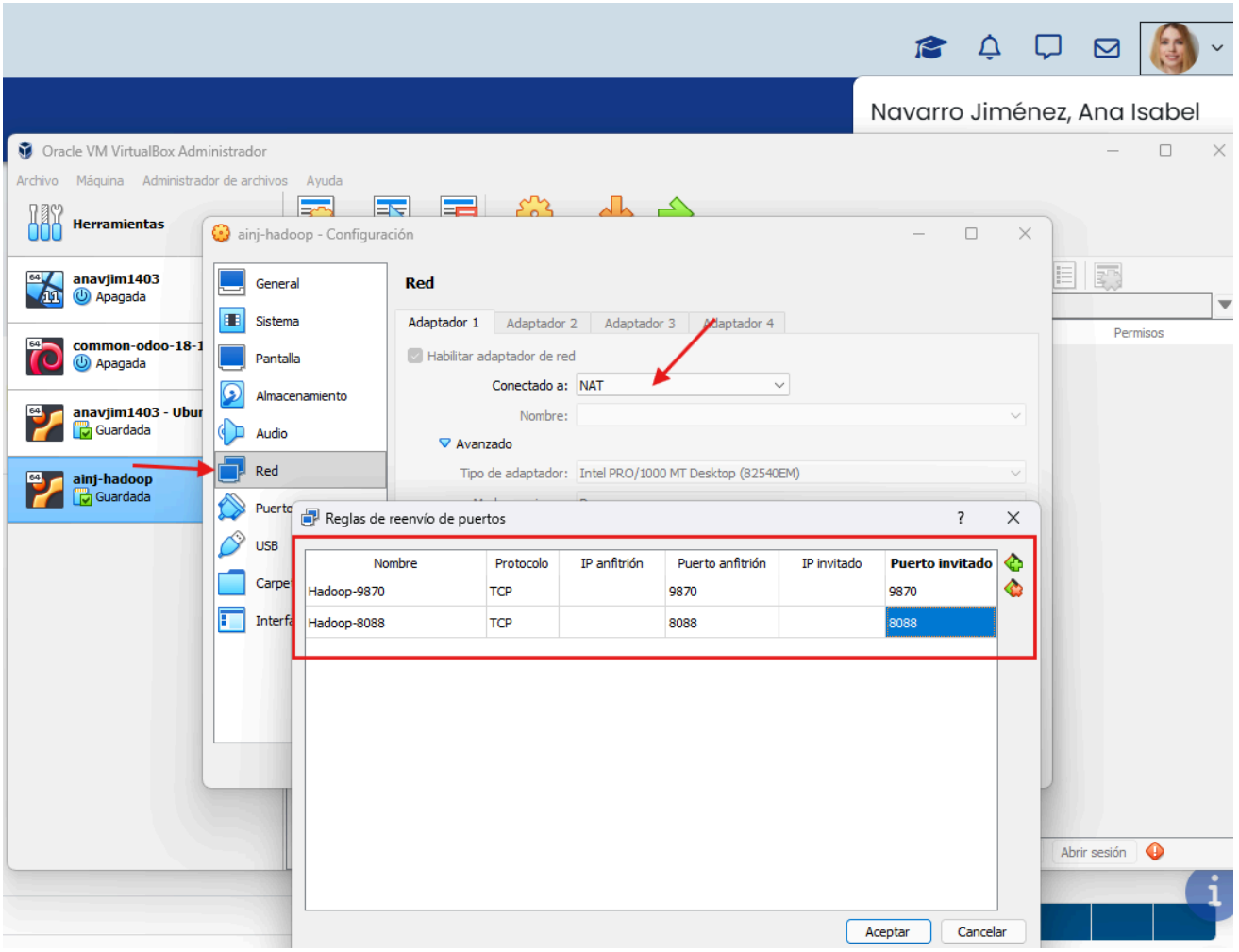
Completamos cada paso con el Nombre y sistema operativo, Hardware y Disco Duro



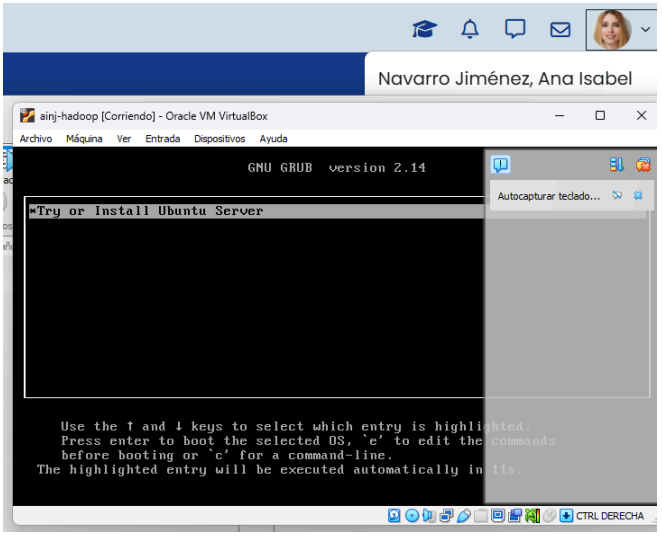
Memoria (Hardware) y Almacenamiento (Disco Duro)



Para la red vamos a modificar en ajustes de la máquina una vez creada con los siguientes datos para permitir el acceso a internet : ***Máquina > Configuración > Red > (Seleccionamos NAT) > (Completamos con las reglas de reenvío siguientes)***



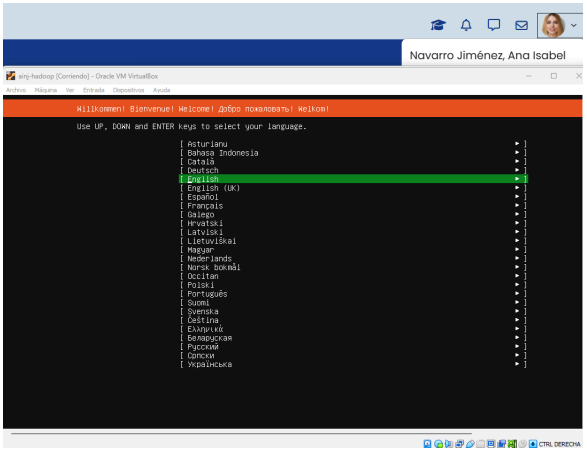
Arranque

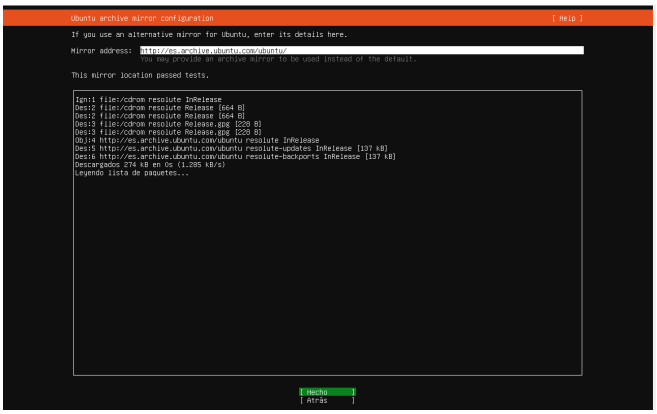


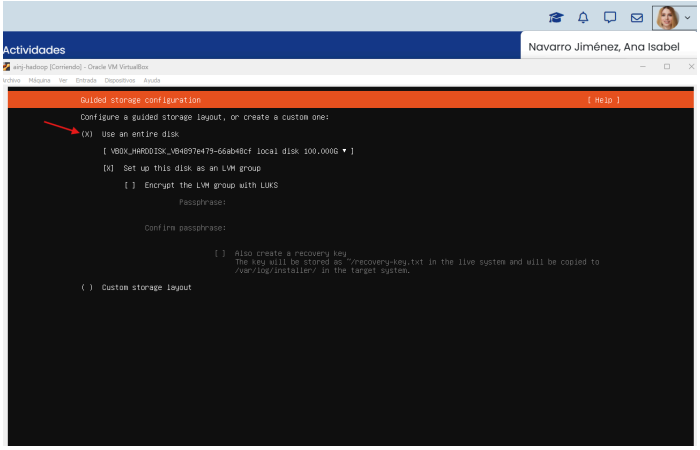
2. Configuración del nombre del equipo y preparación del sistema

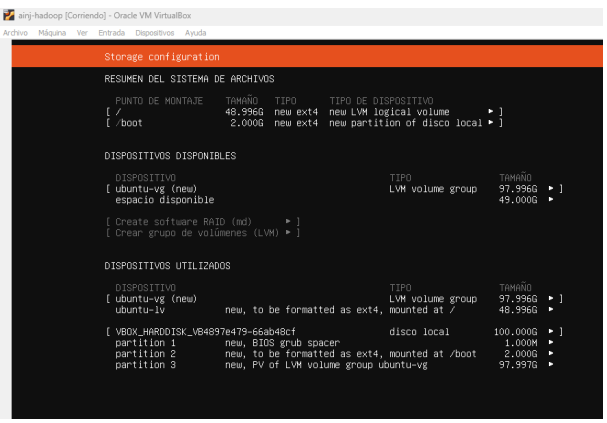
Configura el nombre del nodo con el formato nodo1_iniciales_26_27, compruébalo con hostname y hostnamectl, actualiza el sistema, verifica la conectividad e instala Java. Explica por qué Hadoop necesita Java y la función de JAVA_HOME.

Una vez que hemos creado la máquina procedemos con la instalación, para esta tarea he optado por capturar las imágenes más importantes, el resto de los pasos de la instalación se debe quedar por defecto y pulsar en continuar. El primer paso sería el idioma y continuamos con los siguientes hasta que aparezca el disco







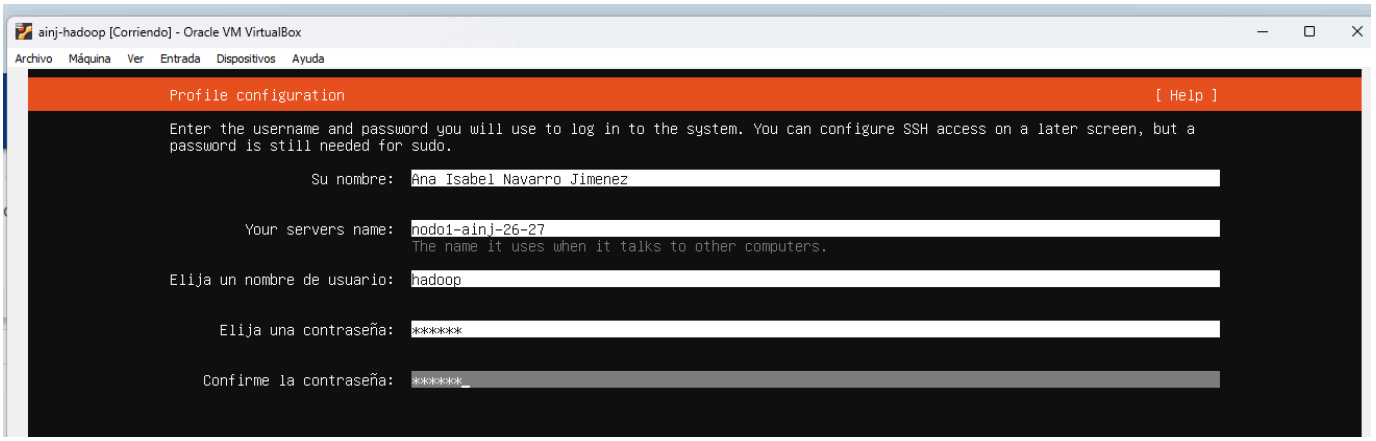


A continuación modificamos el nombre del usuario hadoop como indica la tarea :

Nombre servidor: nodo1-ainj-26-27

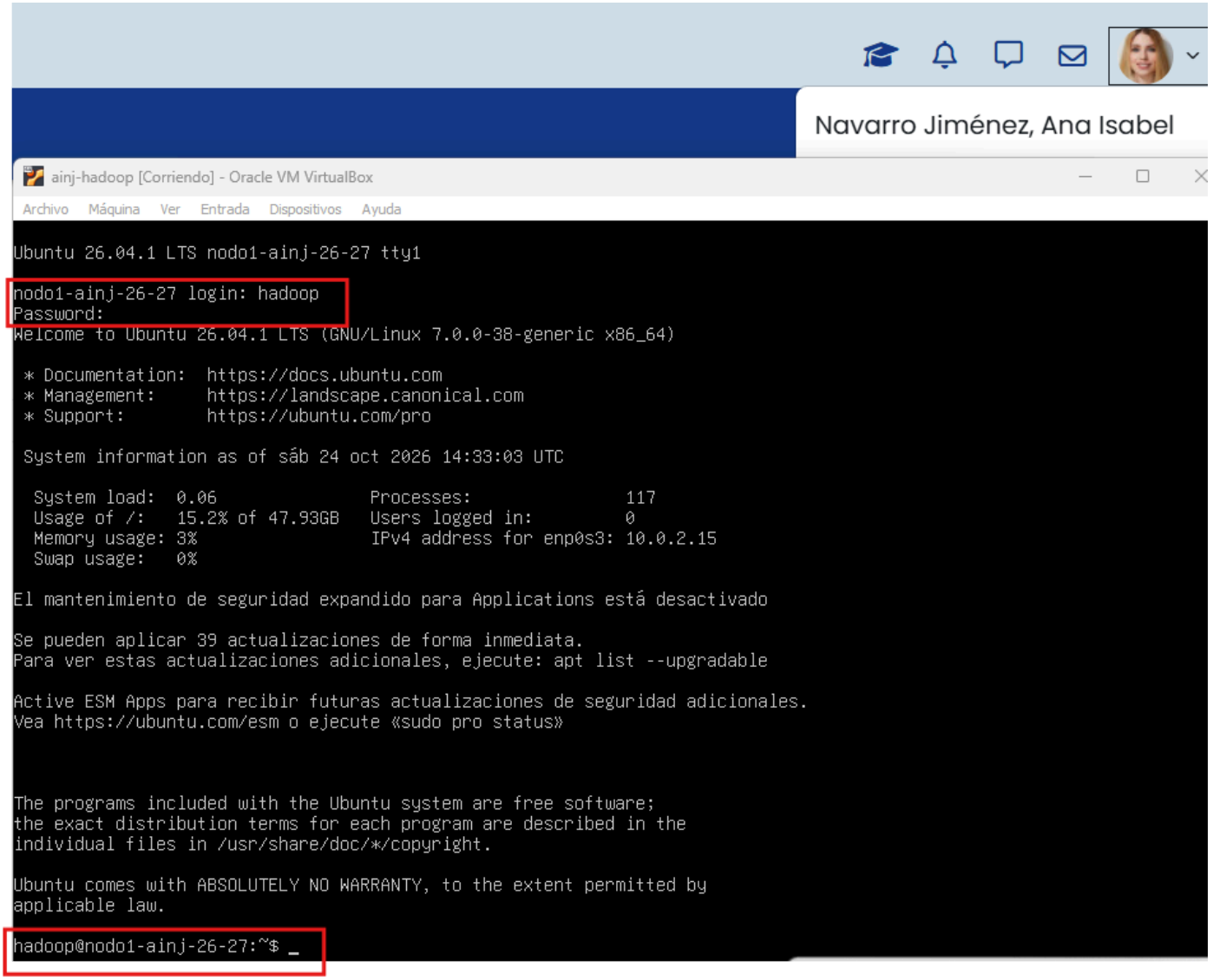
Nombre usuario: hadoop

Contraseña: hadoop

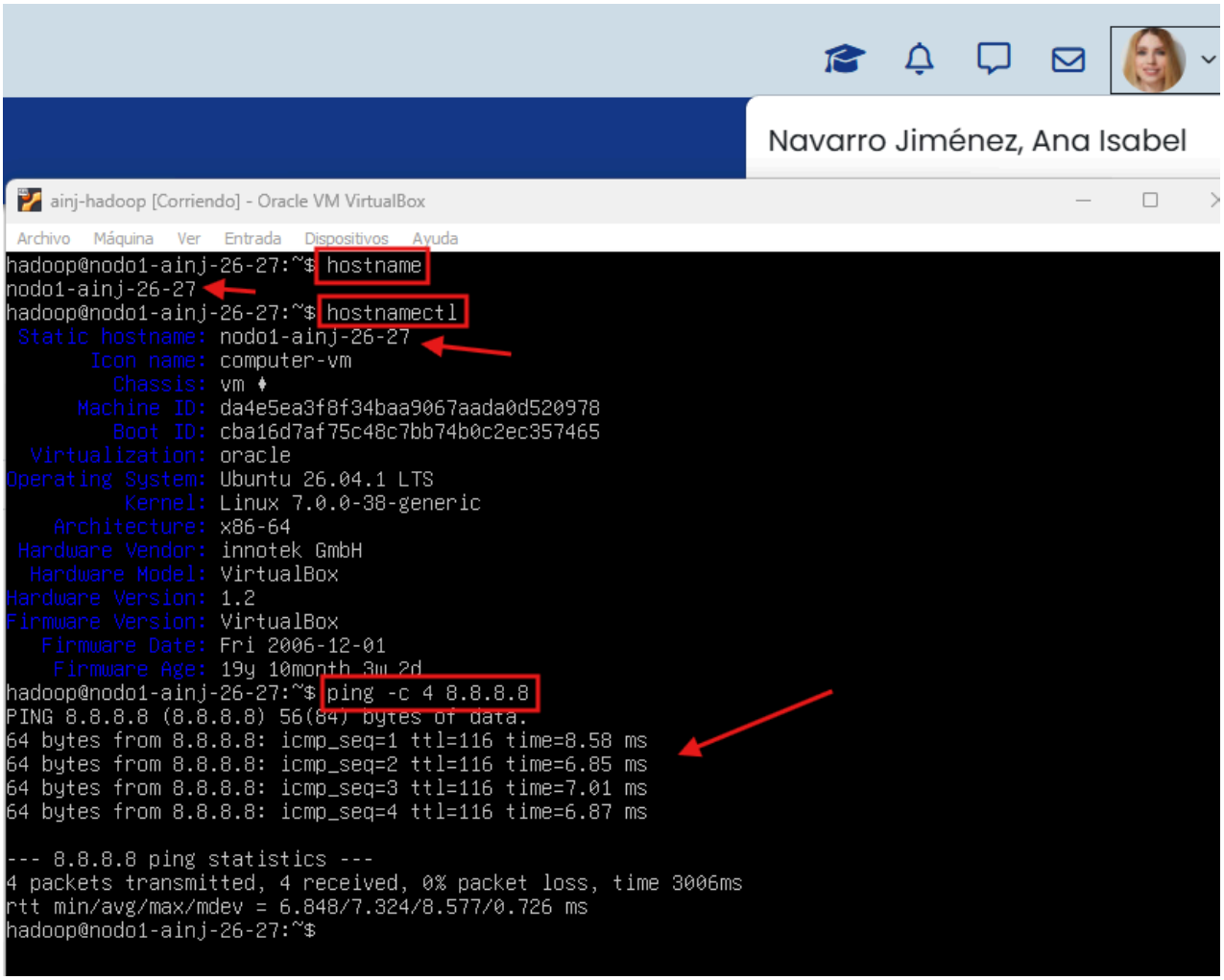


Y terminamos la configuración y el resto de pasos.

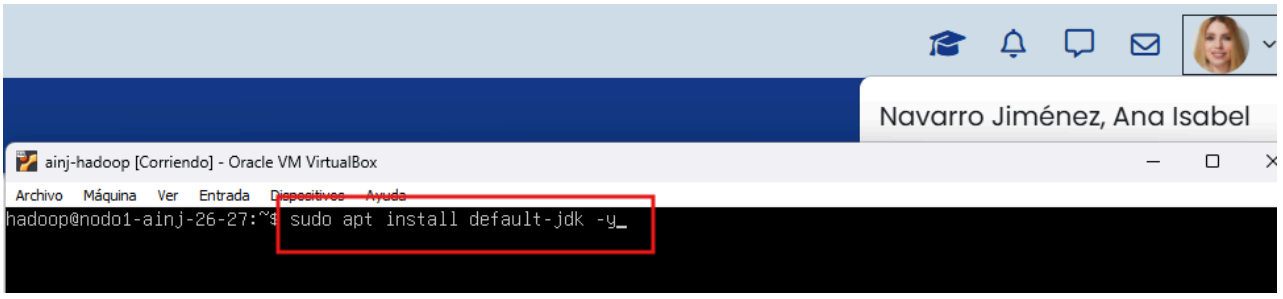
Una vez finalizado ya podemos arrancar hadoop con el usuario ***hadoop*** y contraseña ***hadoop***



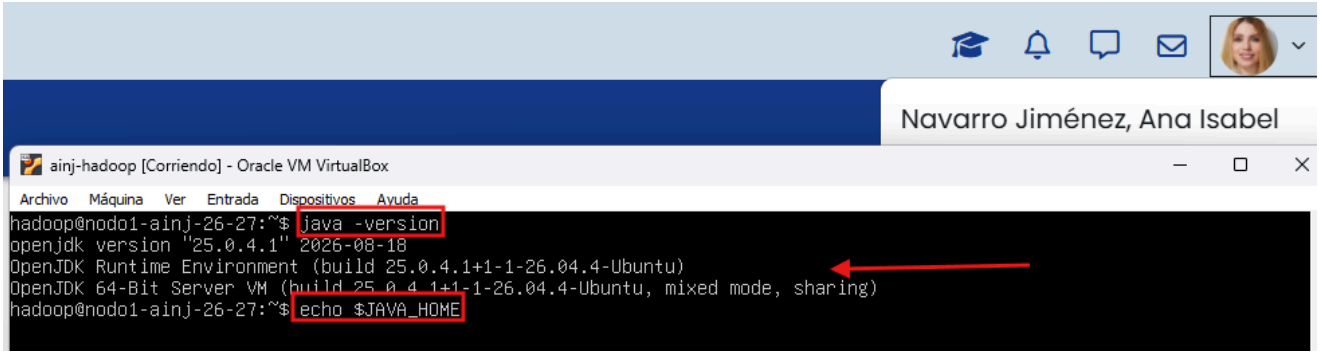
Comprobación con ***hostname > hostnamectl > ping -c 4 8.8.8.8***



Una vez comprobado, instalamos JAVA



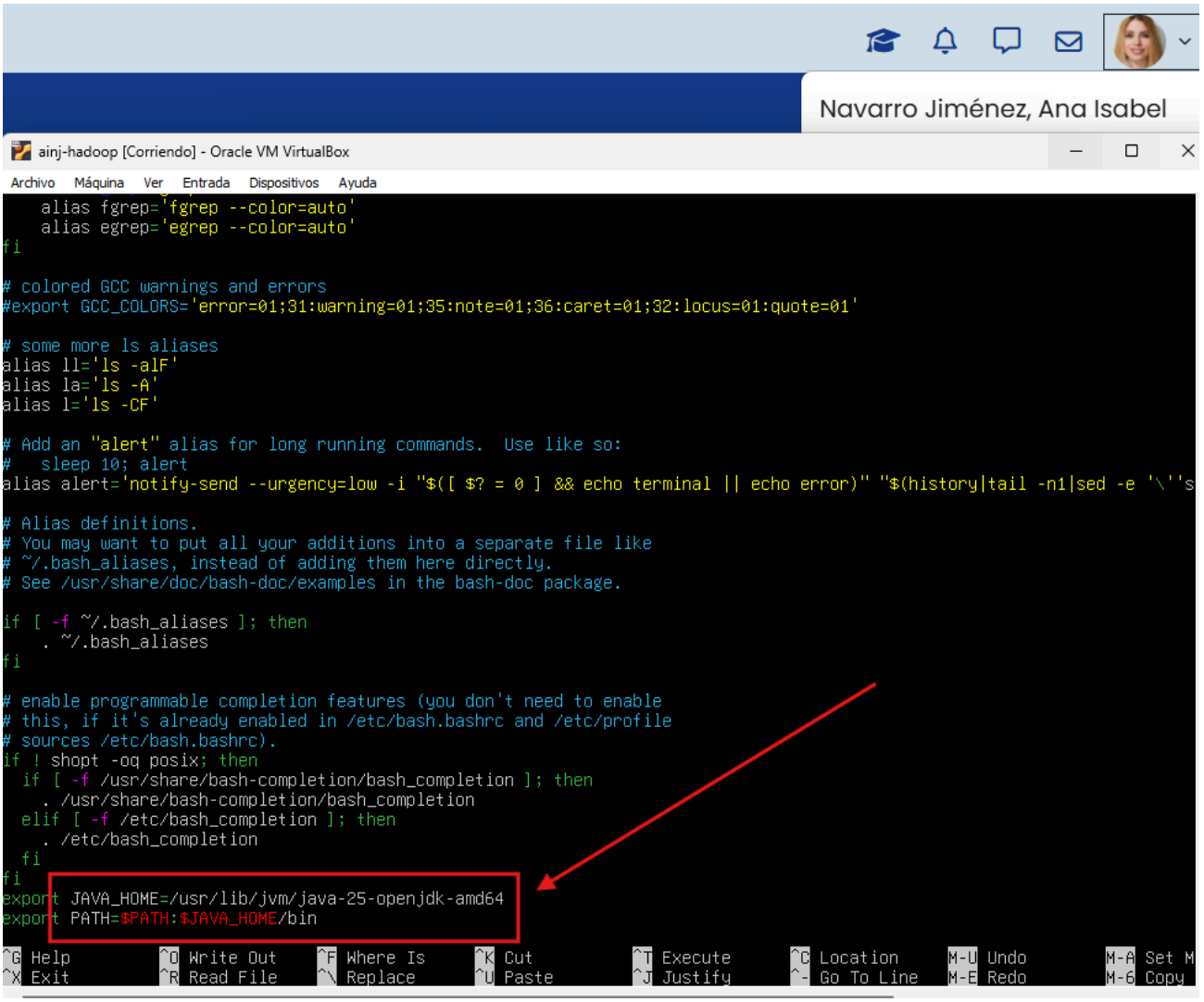
Una vez finalizado comprobamos su instalación: ***java -version > echo $JAVA_HOME***



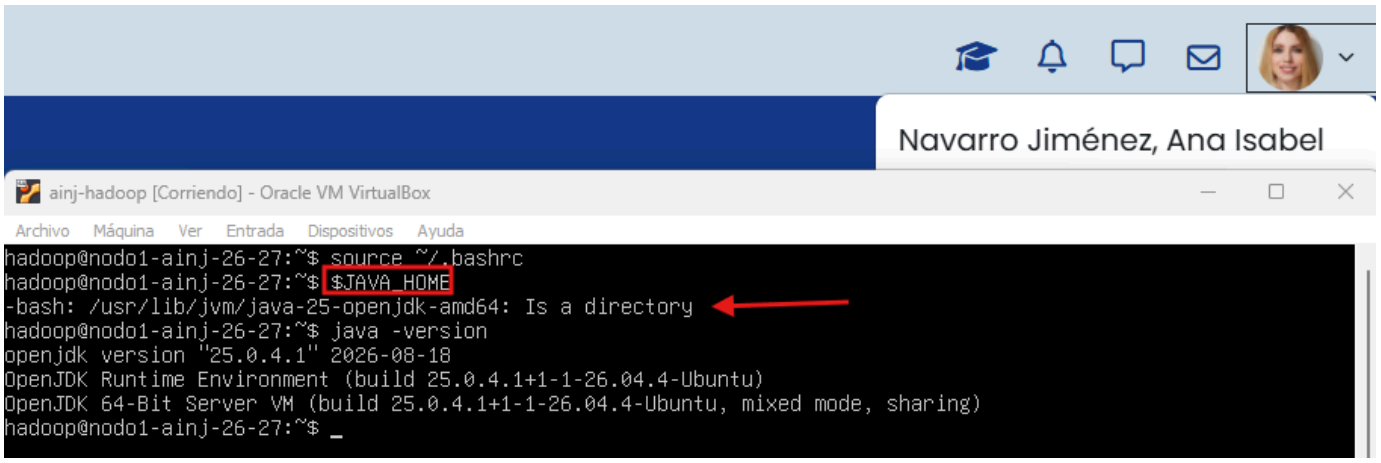
Localizamos donde se encuentra el archivo JAVA_HOME:

```
hadoop@nodo1-ainj-26-27:~$ readlink -f $(which java)
/usr/lib/jvm/java-25-openjdk-amd64/bin/java
hadoop@nodo1-ainj-26-27:~$ nano ~/.bashrc_
```

una vez ahí editamos el archivo con nano, lo exportamos añadiendo las líneas siguientes al final :



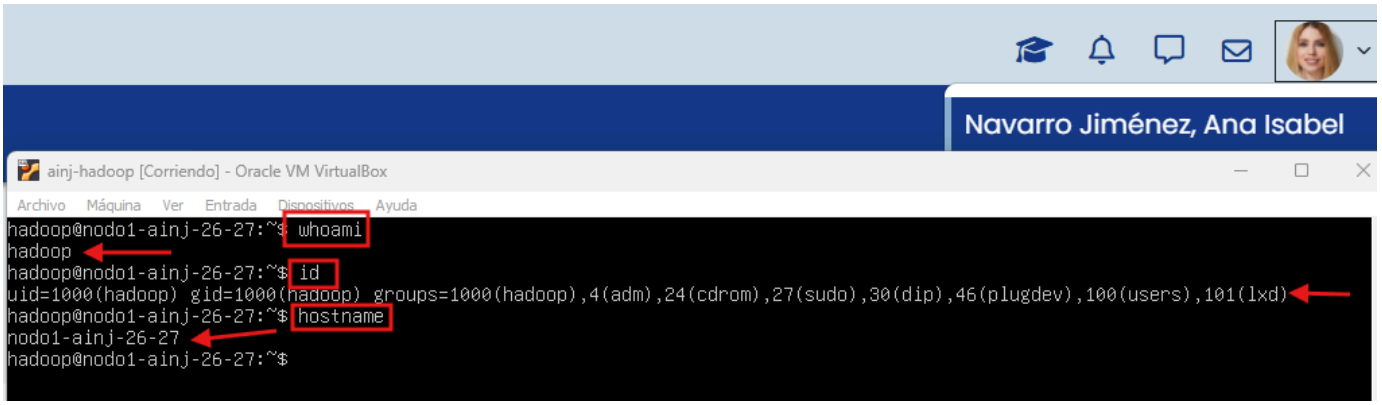
y ya tendremos correctamente configurado el $JAVA_HOME



3. Creación del usuario Hadoop

Crea el usuario hadoop, su directorio personal, configura los permisos necesarios y accede con dicho usuario. El prompt debe tener un formato similar a hadoop@nodo1_abs_26_27:~$.

Como ya configuramos correctamente el usuario en pasos anteriores para confirmarlo añadimos los siguientes comandos : ***whoami > id > hostname***



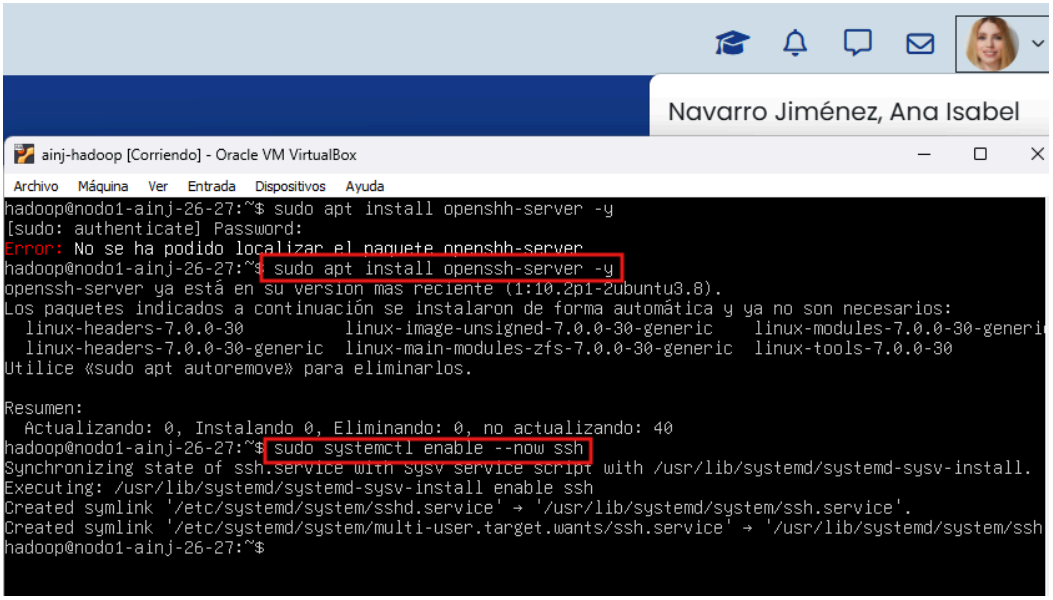
4. Configuración de SSH

Instala y configura SSH para que el usuario hadoop pueda conectarse sin contraseña mediante clave pública. Prueba ssh localhost y ssh nodo1_iniciales_26_27. Explica por qué Hadoop utiliza SSH.

Comprobamos que tenemos instalado el servidor SSH

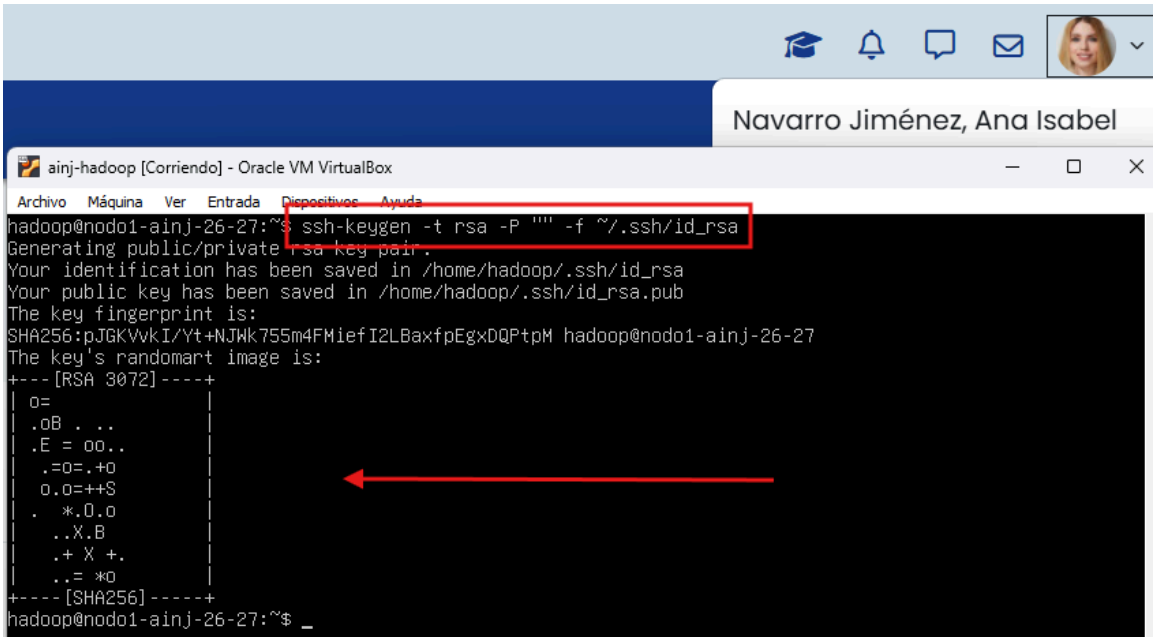
sudo apt install openssh-server -y > sudo systemctl enable --now ssh



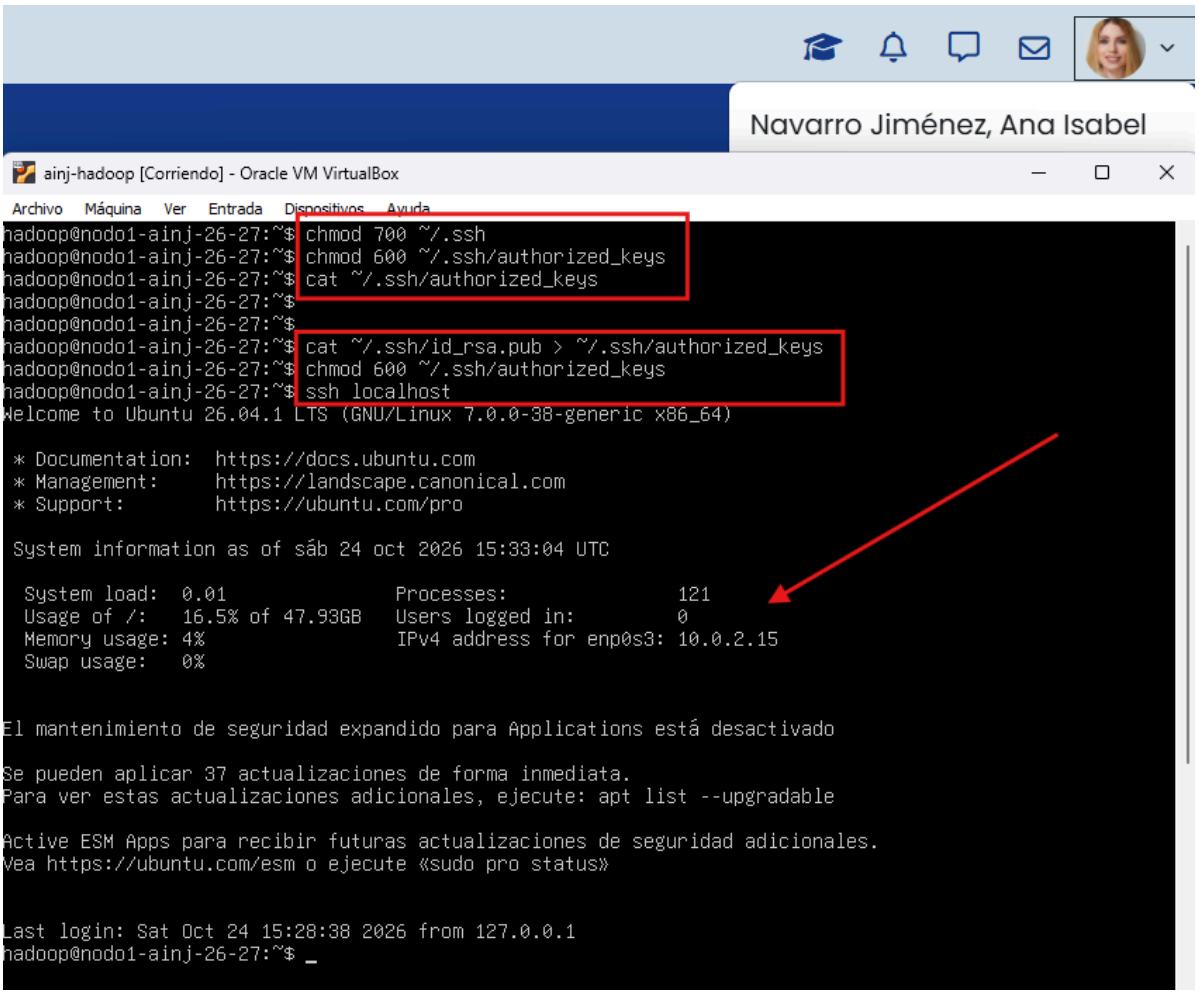


Generamos el par de claves RSA

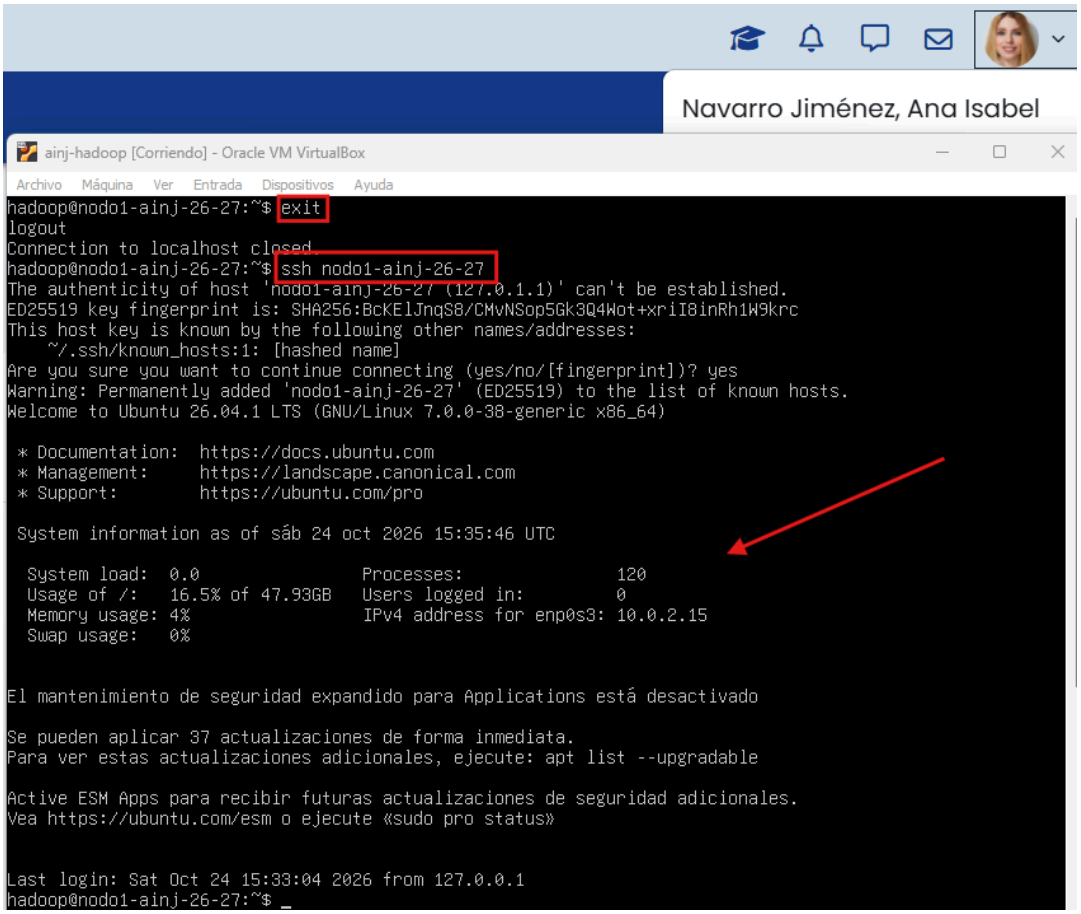
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa



Autorizamos la clave pública para permitir el acceso local, damos los permisos correspondientes y probamos la conexión SSH con ***ssh localhost*** (acceso sin contraseña)



Y para acceder con el hostname , cerramos la sesión anterior y accedemos con ***ssh nodo1-ainj-26-27***

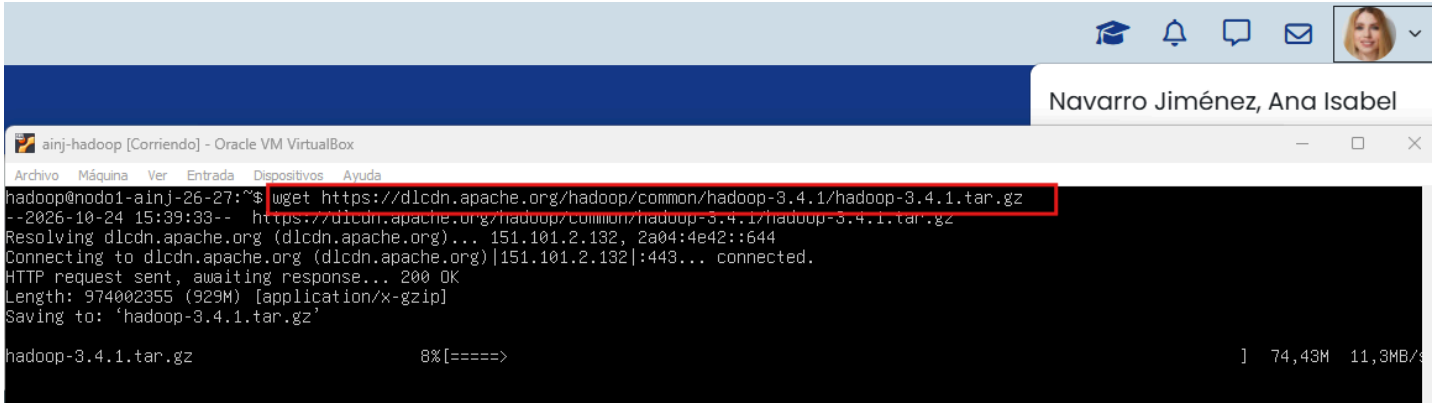


5. Instalación de Apache Hadoop

Descarga e instala Apache Hadoop en /opt/hadoop. Configura JAVA_HOME, HADOOP_HOME, HADOOP_CONF_DIR y PATH. Comprueba la instalación con hadoop version.

Comenzamos descargando y descomprimiendo Apache Hadoop en una versión estable (3.4.1) con los siguientes comandos:

wget https://dlcdn.apache.org/hadoop/common/hadoop-3.4.1/hadoop-3.4.1.tar.gz
sudo tar -xzvf hadoop-3.4.1.tar.gz -C /opt/
sudo mv /opt/hadoop-3.4.1 /opt/hadoop
sudo chown -R hadoop:hadoop /opt/hadoop

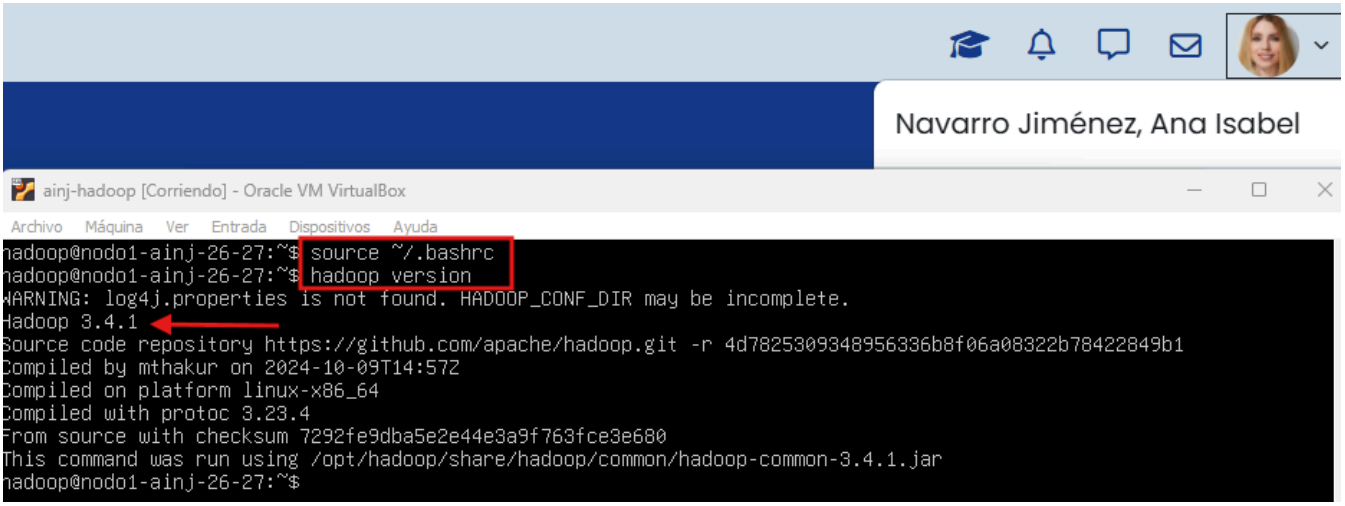


Y una vez realizado configuramos las variables de entorno con ***nano ~/.bashrc***
añadimos las siguientes líneas al final del fichero

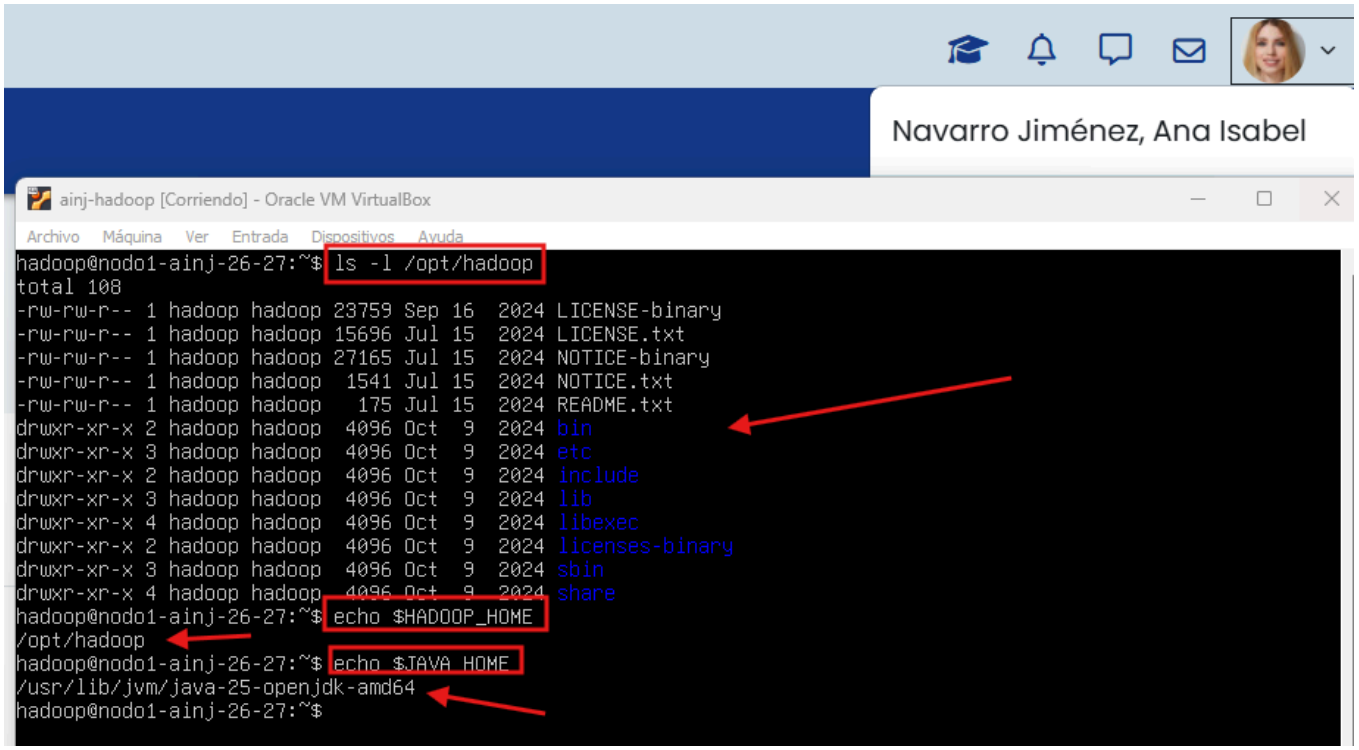
```
  fi
fi
export JAVA_HOME=/usr/lib/jvm/java-25-openjdk-amd64
export PATH=$PATH:$JAVA_HOME/bin

export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/ect/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin_
```

y comprobamos su instalación correcta:



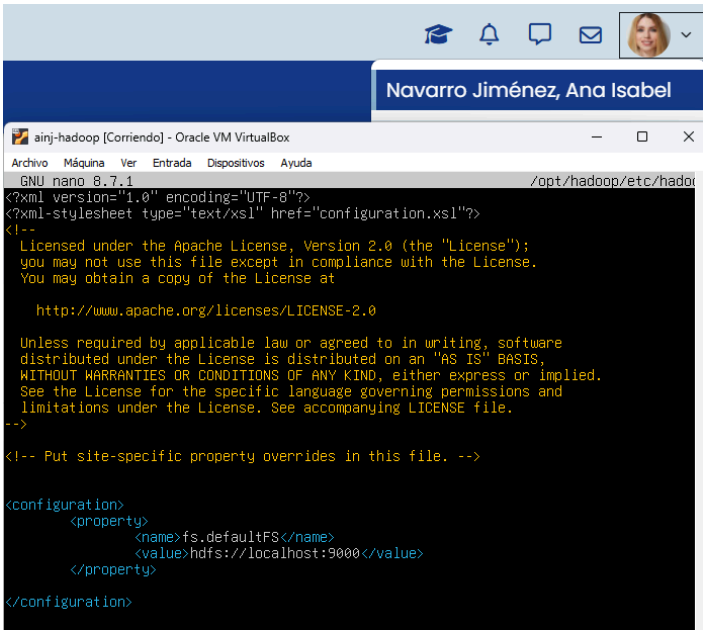
y también en el directorio /opt/hadoop



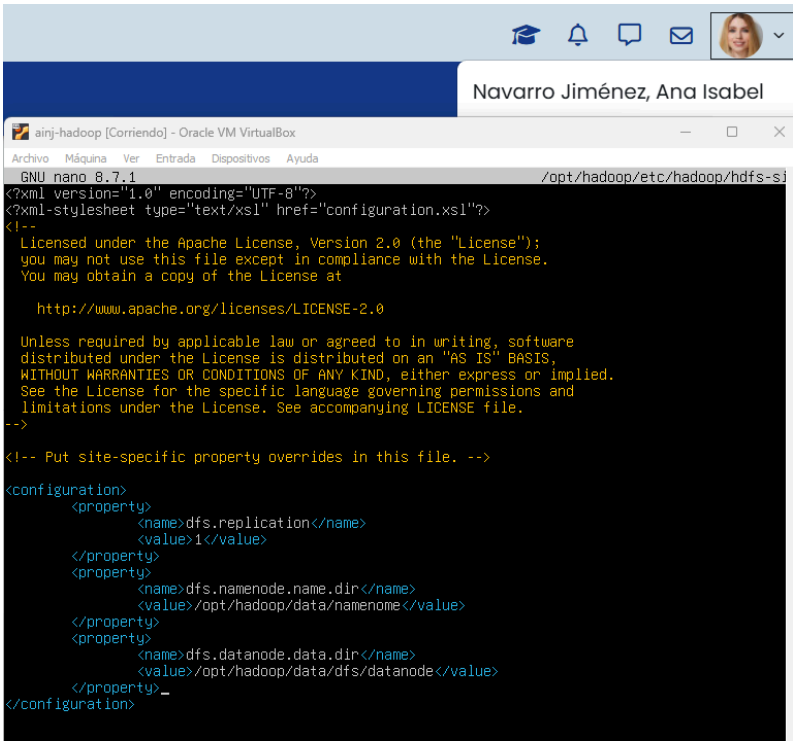
6. Configuración pseudodistribuida

Configura core-site.xml, hdfs-site.xml, mapred-site.xml y yarn-site.xml para trabajar en modo pseudo-distribuido. Explica la función de cada fichero y justifica dfs.replication = 1.

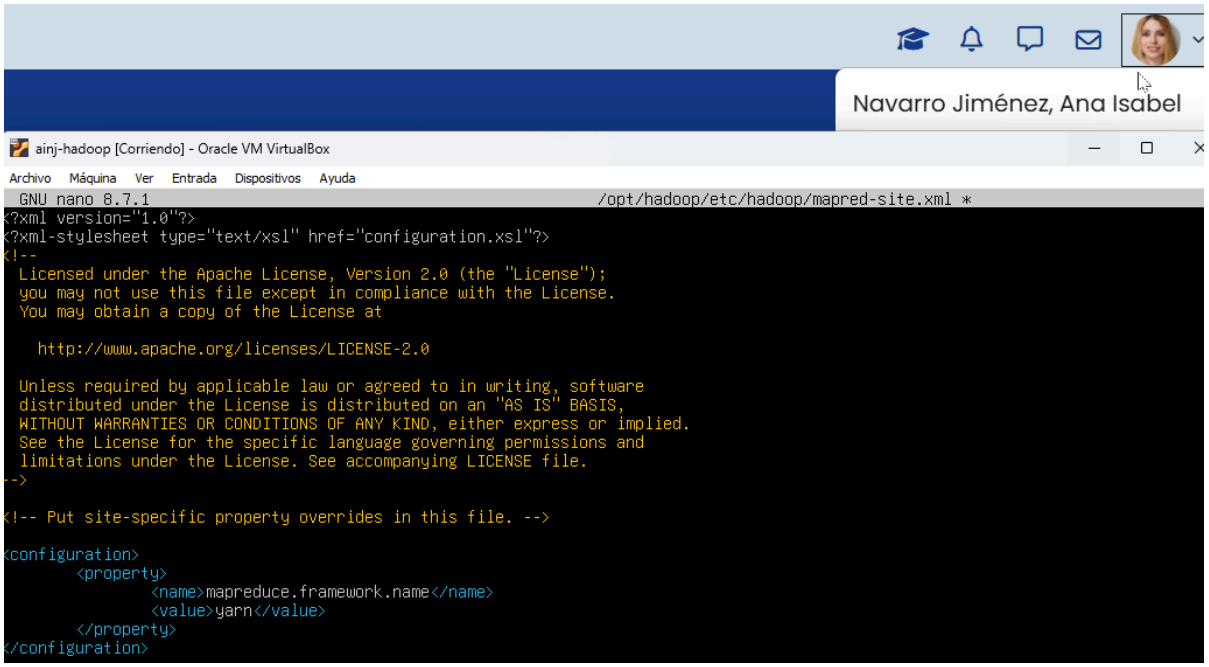
- **Fichero core-site.xml :** Define los parámetros generales del núcleo de Hadoop, como la dirección por defecto del sistema de ficheros distribuido (fs.defaultFS) y los directorios temporales.



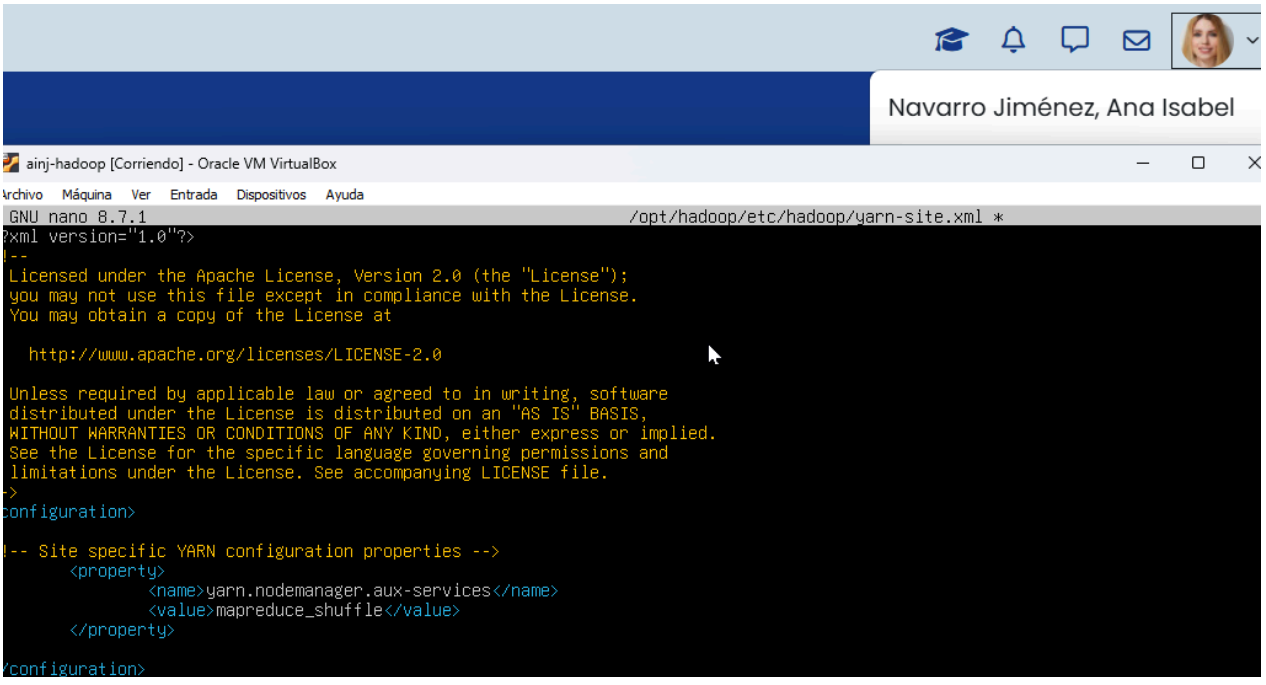
- **Fichero hdfs-site.xml:** Configura los parámetros específicos del servicio HDFS (*Hadoop Distributed File System*), tales como el factor de replicación de los bloques y las rutas locales de almacenamiento físico en el disco para el *NameNode* y los *DataNodes*.



- **Fichero mapred-site.xml:** Configura el framework de procesamiento MapReduce, indicando en las versiones modernas de Hadoop que las tareas de procesamiento deben ejecutarse gestionadas sobre el recurso YARN



- **Fichero yarn-site.xml:** Configura los servicios de gestión de recursos y programación de tareas de YARN (*ResourceManager* y *NodeManagers*), incluyendo los servicios auxiliares necesarios para las fases de ejecución de trabajos (*shuffle*).



Justificación: En una instalación en modo pseudo-distribuido (un único nodo físico o máquina virtual que emula un clúster), todos los demonios (NameNode y DataNode) corren en la misma y única máquina. Por lo tanto, físicamente sólo existe un DataNode disponible. Si configuramos un factor de replicación superior a 1 (por ejemplo, 3), HDFS intentaría replicar cada bloque de datos en tres nodos distintos, lo cual es imposible al no existir otros nodos físicos, generando advertencias o errores en el sistema. Por ello, en un entorno de nodo único, dfs.replication debe establecerse obligatoriamente a 1.

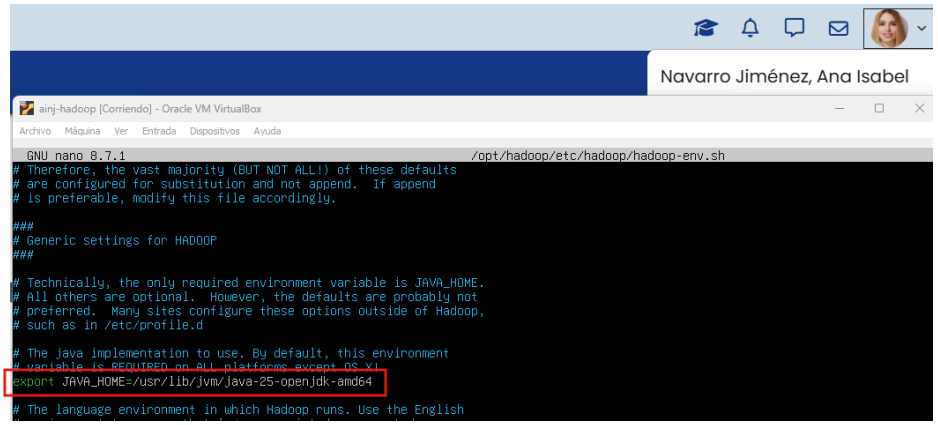
7. Formateo y puesta en marcha de HDFS y YARN

ormatea el NameNode, inicia HDFS y YARN y comprueba los procesos mediante jps. Deben aparecer, según la configuración, NameNode, DataNode, SecondaryNameNode, ResourceManager y NodeManager.

Comenzamos asegurando la variable de java en el entorno de hadoop:

nano /opt/hadoop/etc/hadoop/hadoop-env.sh

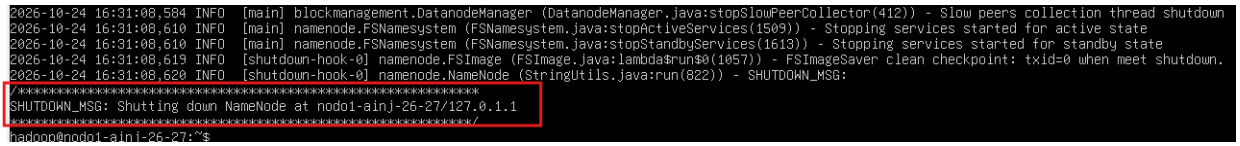
y añadimos el export

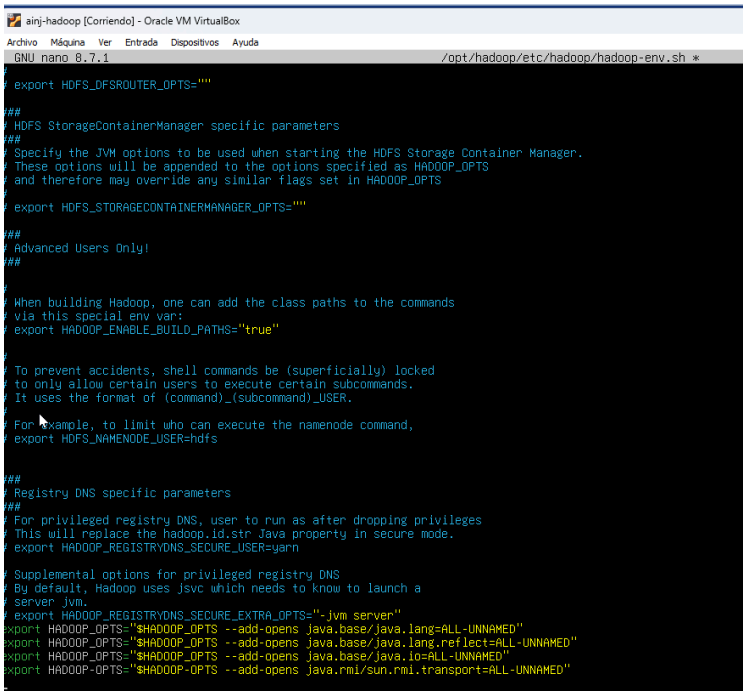


Formateamos el NameNode

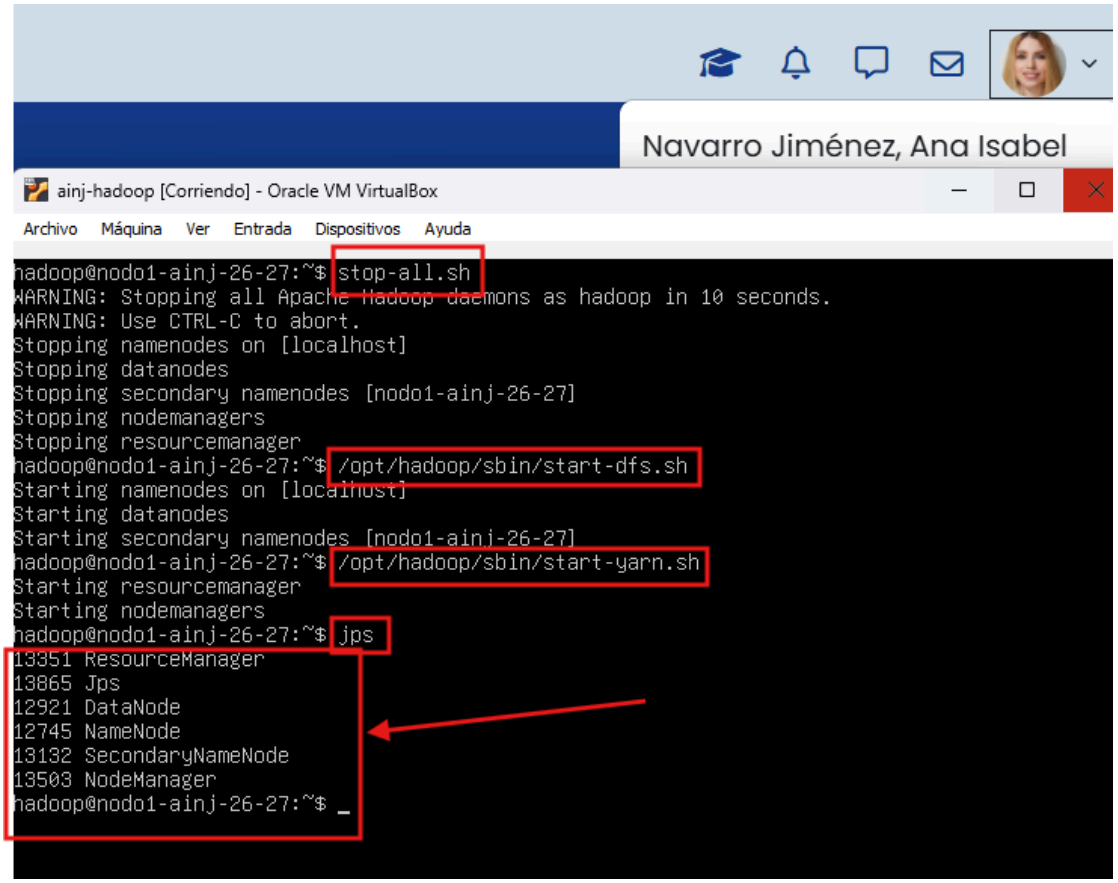
hdfs namenode -format

(*) ANEXO : (En esta parte de la tarea hadoop estaba intentando ejecutar con java 25 que era demasiada moderna, por lo que previamente he tenido que instalar una versión limpia de Java 17 (apt install openjdk-17 -y) y he actualizado también los dos ficheros de JAVA_HOME con la nuevas rutas) y ya si funcionó





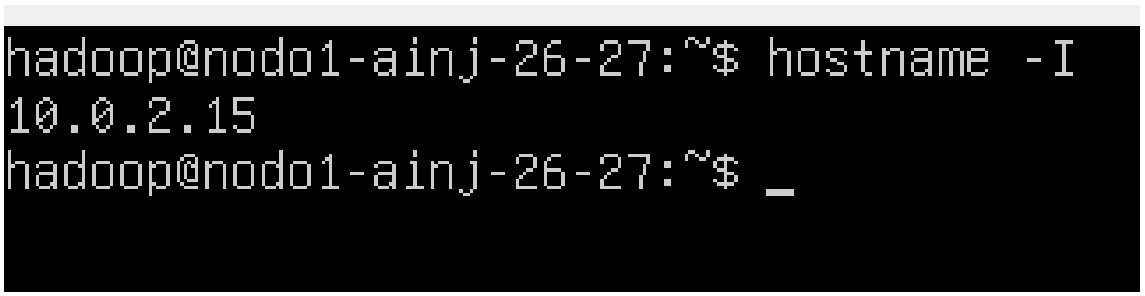
Es un problema clásico de compatibilidad entre Hadoop 3.4.1 y Java 17. A partir de Java 9 (y de forma estricta en Java 17), el sistema de módulos (JPMS) prohíbe por defecto que librerías antiguas (como las de inyección de dependencias y reflection que usa YARN en su arranque) accedan a métodos internos protegidos del paquete java.lang a menos que se lo permitamos explícitamente mediante parámetros de la máquina virtual de Java (JVM).



8. Interfaces web de Hadoop

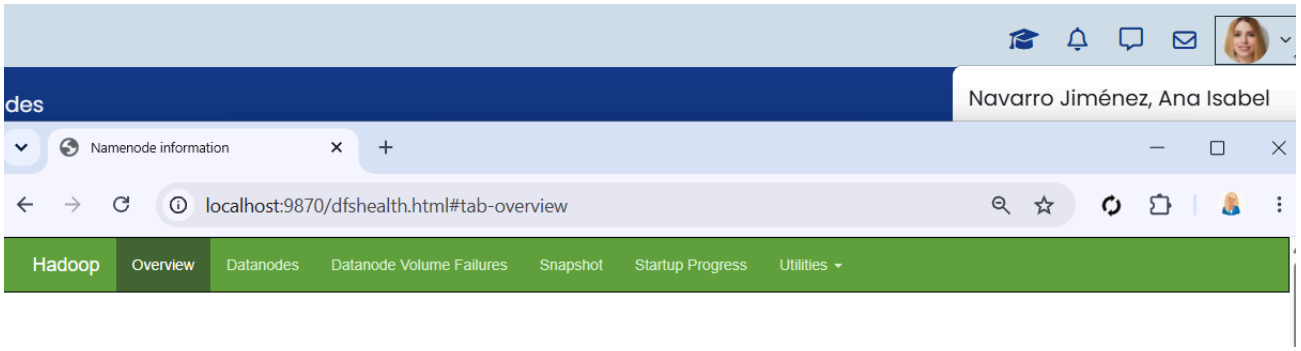
Accede a la interfaz del NameNode en http://IP_DEL_NODO:9870 y a YARN ResourceManager en http://IP_DEL_NODO:8088. Explica brevemente qué información ofrece cada interfaz.

Localizamos la ip:



1. **Interfaz web del NameNode**

- **Función e información que ofrece:** Es el panel de control principal de HDFS (Hadoop Distributed File System). Proporciona una visión general y en tiempo real del estado del almacenamiento. Muestra métricas clave como la capacidad total del disco, el espacio libre y usado, el porcentaje de almacenamiento ocupado, el número de ficheros y bloques activos, y la lista de DataNodes conectados para comprobar la salud del clúster. Además, incluye una pestaña ***(Utilities -> Browse the file system)*** que permite explorar los directorios y ficheros almacenados en HDFS de forma visual.



Overview 'localhost:9000' (active)

Started:	Sat Oct 24 21:13:57 +0200 2026
Version:	3.4.1, r4d7825309348956336b8f06a08322b78422849b1
Compiled:	Wed Oct 09 16:57:00 +0200 2024 by mthakur from branch-3.4.1
Cluster ID:	CID-1f379b8a-86e9-4f06-b864-792172e35e33
Block Pool ID:	BP-948150805-127.0.1.1-1792867522305

Summary

Security is off.

Safemode is off.

1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).

Heap Memory used 79.23 MB of 129 MB Heap Memory. Max Heap Memory is 1.81 GB.

Non Heap Memory used 52.87 MB of 55.25 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	47.93 GB
Configured Remote Capacity:	0 B
DFS Used:	32 KB (0%)
Non DFS Used:	10.8 GB
DFS Remaining:	34.66 GB (72.32%)
Block Pool Used:	32 KB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Sat Oct 24 21:13:57 +0200 2026
Last Checkpoint Time	Sat Oct 24 20:45:22 +0200 2026
Last HA Transition Time	Never
Enabled Erasure Coding Policies	RS-6-3-1024k

NameNode Journal Status

Current transaction ID: 5

Journal Manager	State
FileJournalManager(root=/opt/hadoop/data/namenome)	EditLogFileOutputStream(/opt/hadoop/data/namenome/current/edits_inprogress_0000000000000000005)

NameNode Storage

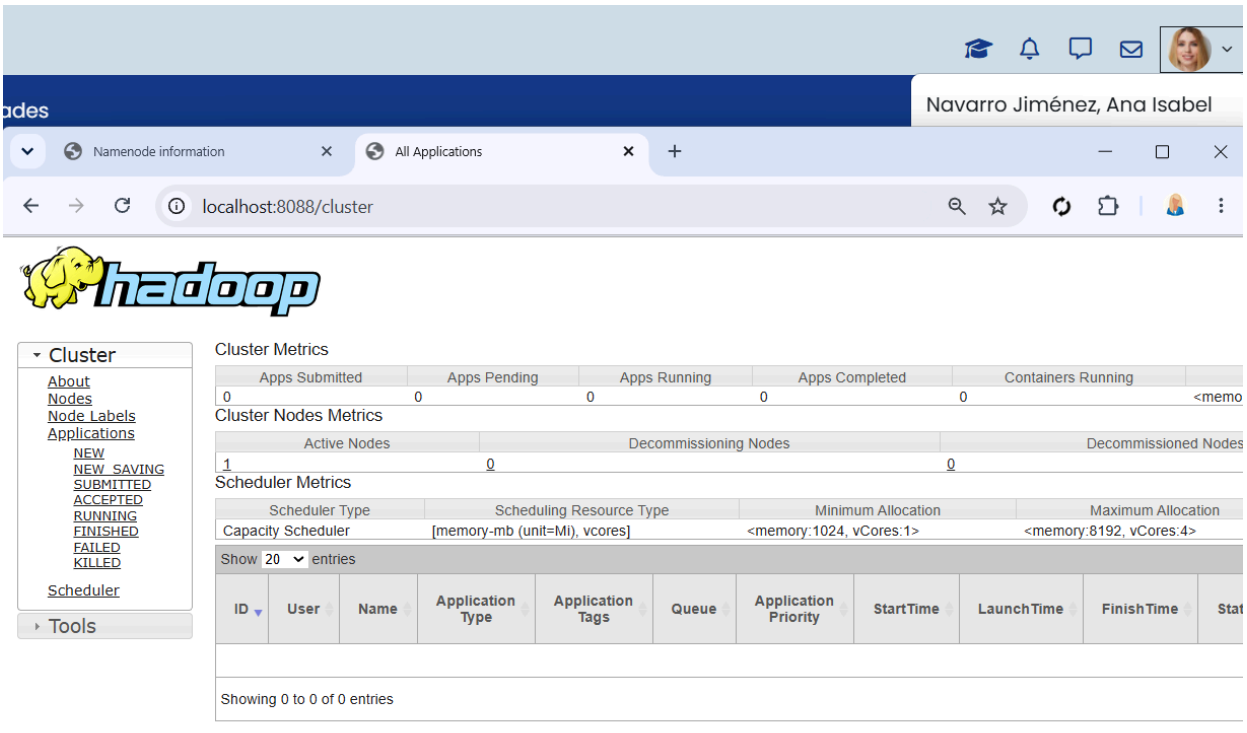
Storage Directory	Type	State
/opt/hadoop/data/namenome	IMAGE_AND_EDITS	Active

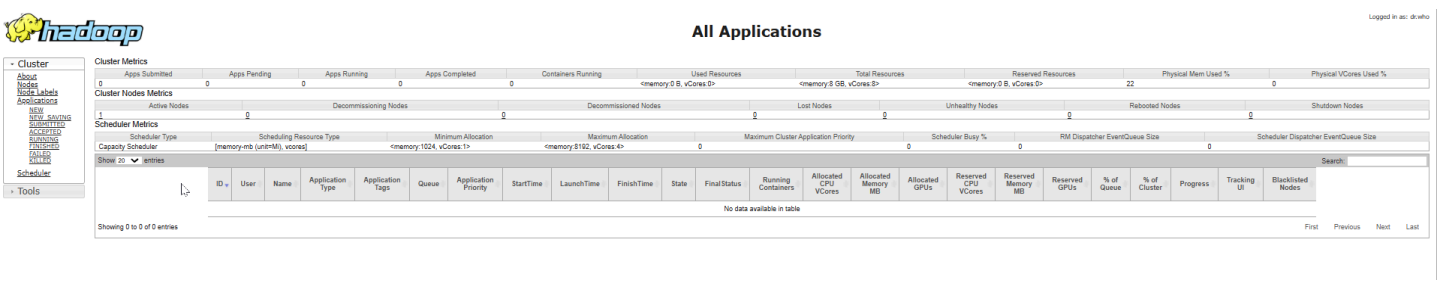
DFS Storage Types

Storage Type	Configured Capacity	Capacity Used	Capacity Remaining	Block Pool Used	Nodes In Service
DISK	47.93 GB	32 KB (0%)	34.66 GB (72.32%)	32 KB	1

2. Interfaz web de YARN ResourceManager

- Función e información que ofrece: Es el panel de gestión y programación de recursos del clúster. Muestra el estado del programador (Scheduler), los nodos administradores activos (NodeManagers), las aplicaciones o trabajos (Jobs) que se están ejecutando en ese momento, el historial de tareas finalizadas, y las métricas detalladas de consumo de memoria RAM y núcleos de CPU en todo el clúster.





9. SIstemas de almacenamiento

Redacta una explicación de 300-500 palabras sobre HDFS, NameNode, DataNode, bloques, replicación, diferencias con un sistema de ficheros tradicional, escalabilidad y ventajas de aumentar el número de nodos. Relaciónalo con la instalación realizada.

Para comprender el funcionamiento de Hadoop, partí de su base fundamental: HDFS (Hadoop Distributed File System), un sistema de ficheros distribuido diseñado para almacenar volúmenes masivos de datos (Big Data) con alta tolerancia a fallos. Frente a un sistema de ficheros tradicional (como EXT4 o NTFS), que gestiona la información de forma local y monolítica en un único disco, pude comprobar cómo HDFS fragmenta los grandes ficheros en unidades lógicas de tamaño fijo llamadas bloques, distribuyéndolos de forma cooperativa.

Esta arquitectura cuenta a través de los componentes principales que levanté y validé en mi máquina virtual:

- **NameNode:** El nodo maestro encargado de gestionar los metadatos. Durante la puesta en marcha, tras definir su ruta de almacenamiento en ***/opt/hadoop/data/dfs/namenode*** dentro del fichero hdfs-site.xml, necesité inicializarlo ejecutando el comando hdfs namenode -format. Este proceso estructuró con éxito el directorio y generó el identificador de bloques inicial, sin el cual los servicios se habrían negado a arrancar.
- **DataNode:** El nodo esclavo responsable de almacenar los bloques de datos reales en el disco local (/opt/hadoop/data/dfs/datanode). En mis primeras pruebas —donde el comando jps se quedaba a medias mostrando sólo algunos demonios—, comprobé que el DataNode y el SecondaryNameNode responden correctamente gracias al script start-dfs.sh, mientras que los demonios de YARN me exigieron depurar los ficheros de configuración.

Asimismo, un concepto crítico que apliqué y sufrí en primera persona fue el sistema de replicación. En un entorno de producción real, cada bloque se duplica en varios nodos para evitar pérdidas de información. Sin embargo, tal como justificé en mi instalación en modo pseudo-distribuido (donde todos los servicios corren en una única máquina virtual y solo existe físicamente un DataNode), me vi obligada a configurar explícitamente el parámetro dfs.replication = 1 en el fichero hdfs-site.xml. De haber mantenido el valor por defecto de un clúster distribuido (como 3), HDFS habría intentado buscar nodos inexistentes, generando fallos en el sistema.

De hecho, esta relación práctica se hizo evidente cuando afronté los errores de compatibilidad tras fallar el arranque de YARN debido a restricciones del sistema de módulos de Java 17 (que bloqueaba las clases internas por seguridad), tuve que añadir los parámetros ***--add-opens*** en el fichero ***hadoop-env.sh***. Esto me permitió ver finalmente los 5 procesos obligatorios (NameNode, DataNode, SecondaryNameNode, ResourceManager y NodeManager) con ***jps***, garantizando que tanto el almacenamiento HDFS como la gestión de recursos estaban sincronizados.

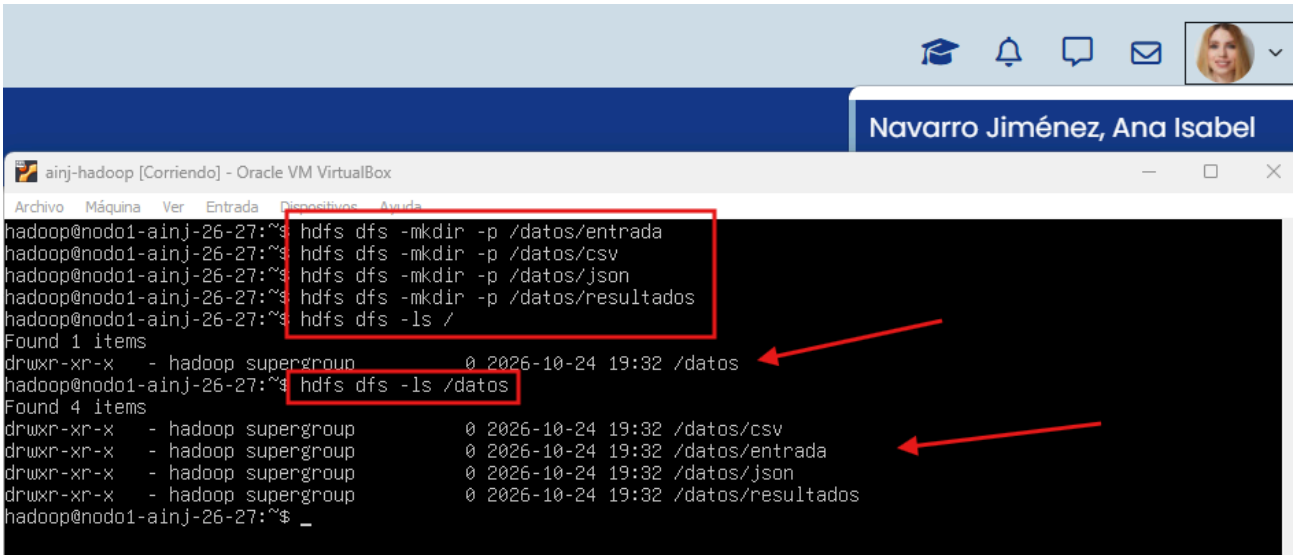
Por último, comprendí que la gran ventaja de HDFS frente a los sistemas tradicionales radica en su escalabilidad horizontal ya que mientras que un sistema tradicional requiere ampliar verticalmente los recursos de un único servidor (con límites físicos infranqueables y altos costes), HDFS permite incrementar de forma masiva la capacidad de almacenamiento y la potencia de procesamiento simplemente añadiendo nuevos nodos DataNode al clúster de manera dinámica. Aumentar el número de nodos multiplica las rutas de acceso en paralelo a los bloques, reduciendo los cuellos de botella y optimizando la ejecución de tareas de procesamiento distribuido (como las que gestiona el ResourceManager que validé en el puerto 8088), cerrando así con éxito todo el ciclo de despliegue de mi actividad.

10. Creación de la estructura HDFS

Crea en HDFS /datos, /datos/entrada, /datos/csv, /datos/json y /resultados. Comprueba la estructura con hdfs dfs -ls / y hdfs dfs -ls /datos.

Creamos los directorios
hdfs dfs -mkdir -p /datos/entrada
hdfs dfs -mkdir -p /datos/csv

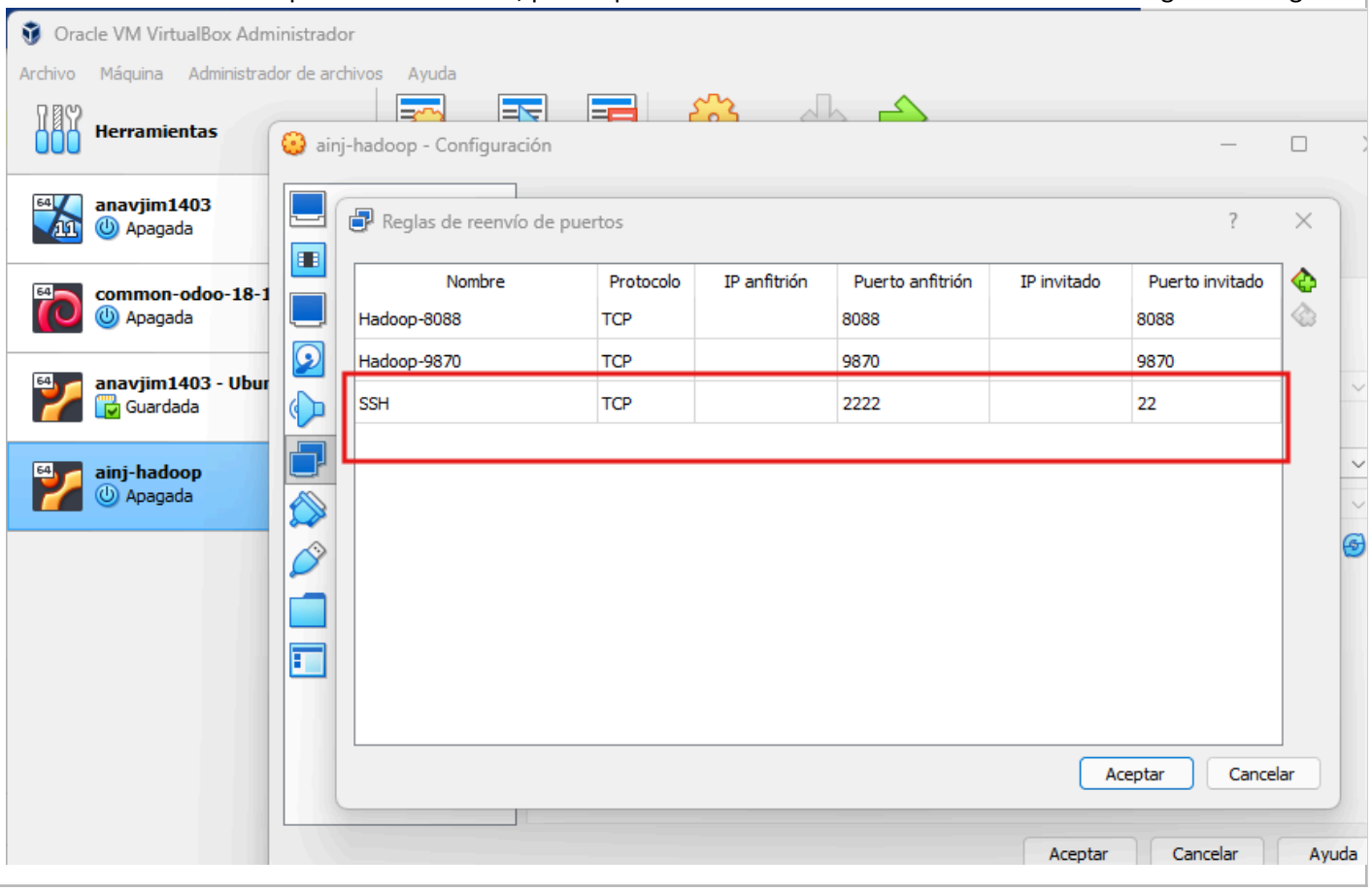
hdfs dfs -mkdir -p /datos/json
hdfs dfs -mkdir -p /resultados



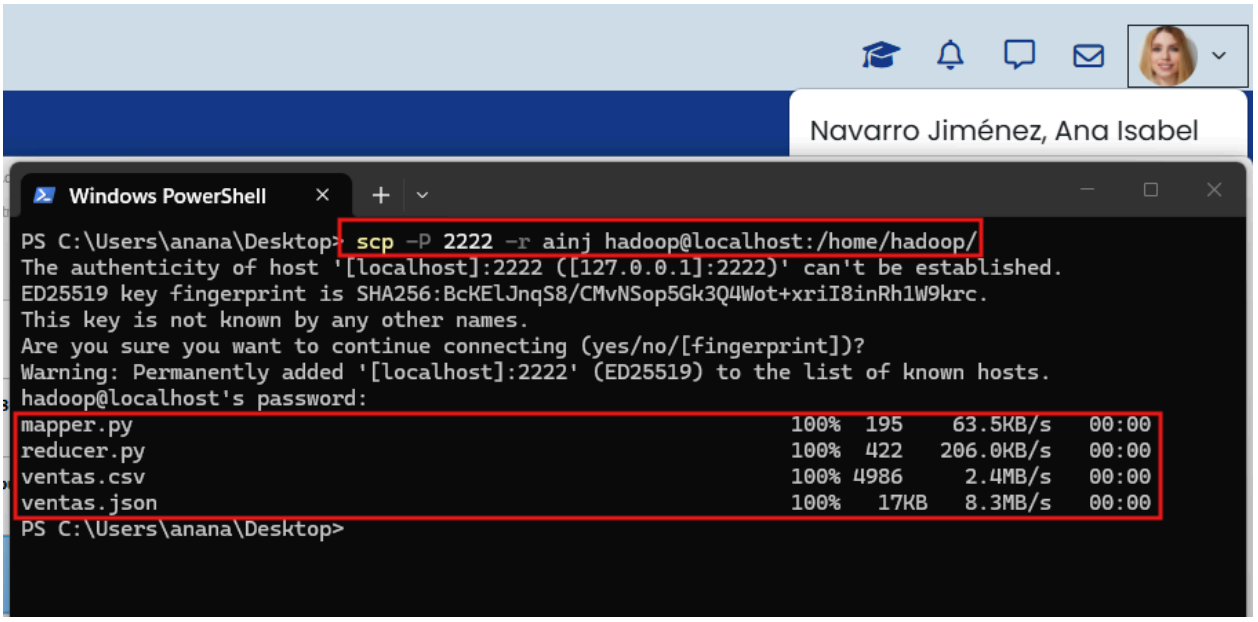
11. Preparación del conjunto de datos

Utiliza el fichero ventas.csv suministrado, que contiene 100 registros más una cabecera. Identifica sus campos y comprueba el número de líneas mediante wc -l ventas.csv.

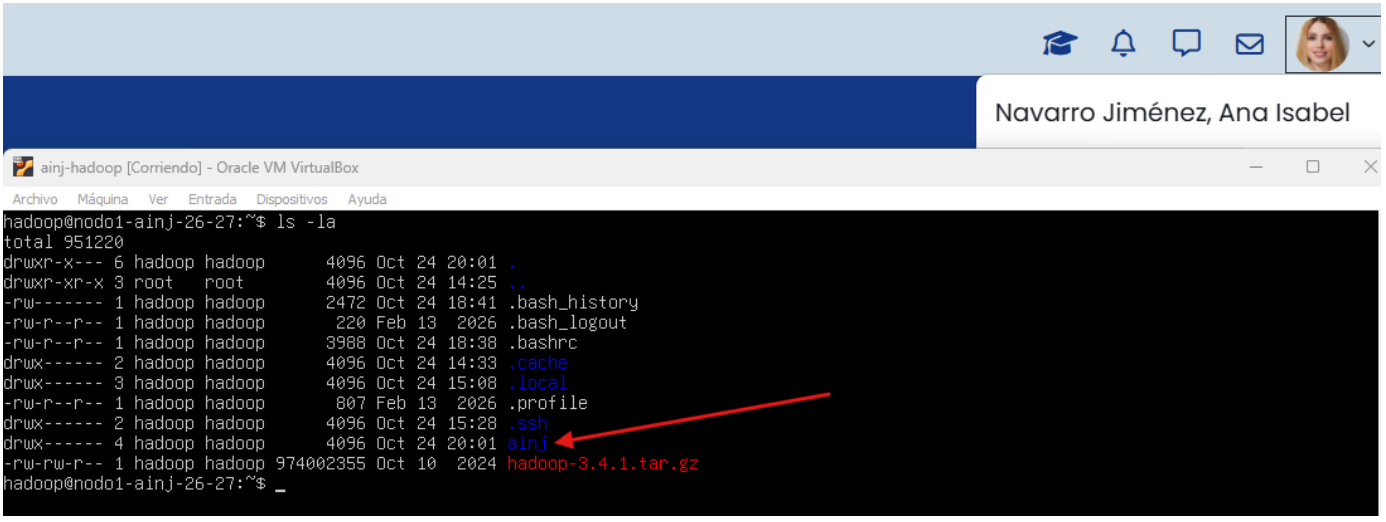
Para poder operar con los datos he creado una carpeta en mi escritorio y vamos a pasarlo a la máquina virtual con todos los archivos que vamos a utilizar , por lo que añadimos una nueva conexión ssh con los siguientes reglas



y ahora en nuestro equipo podemos ejecutar el comando desde powershell para copiar la carpeta con los archivos a la máquina



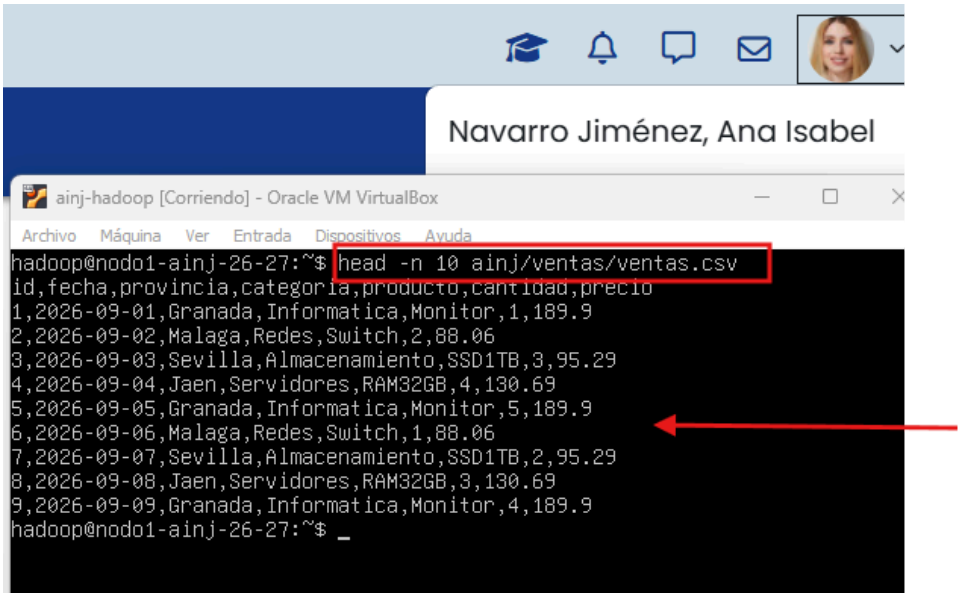
y con el comando ls -la podemos ver si se ha enviado correctamente la carpeta (ainj)



y ya podemos ejecutar el comando para ver el número de filas:



Y para ver parte del contenido:

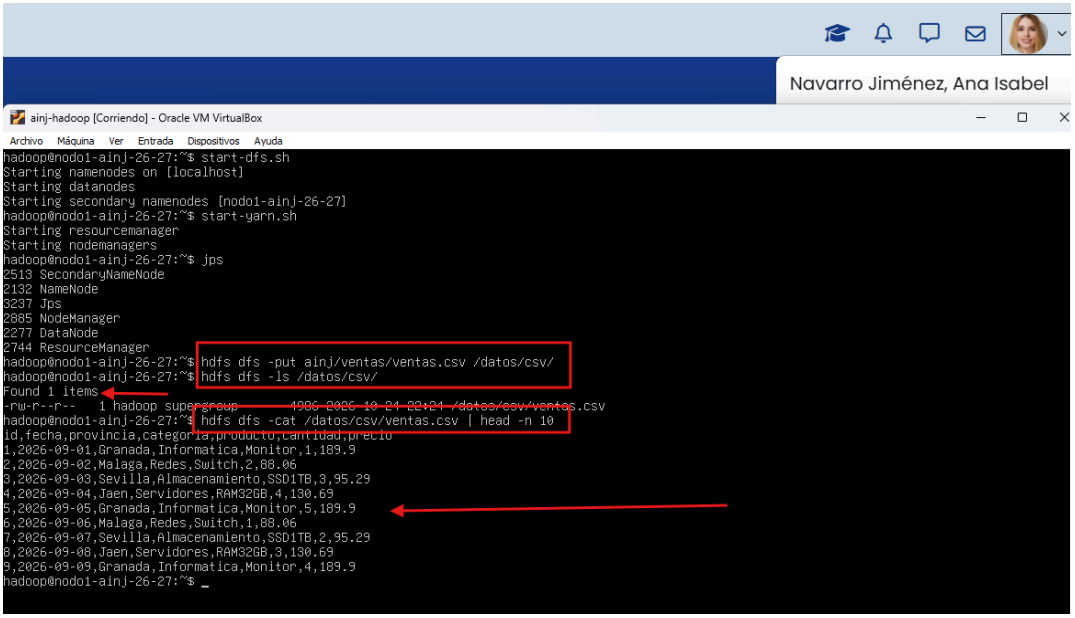


12. Ingestión de datos en HDFS

Realiza la ingestión del fichero local en HDFS usando y explicando comandos como hdfs dfs -put, hdfs dfs -copyFromLocal, hdfs dfs -ls y hdfs dfs -cat. Explica la diferencia entre put y copyFromLocal.

Realizamos la ingesta con ***hdfs dfs -put*** , comprobamos con ***hdfs sfd -ls*** y visualizamos el contenido con ***-cat***

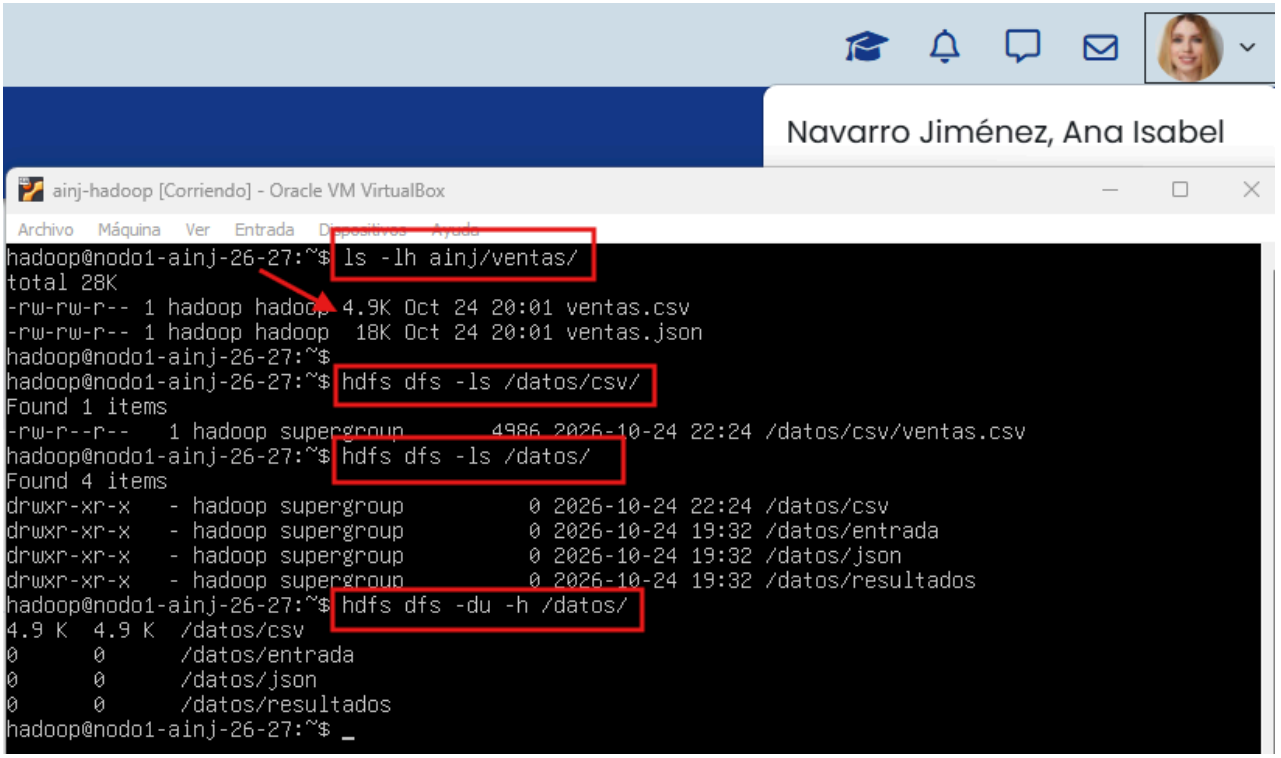
- **hdfs dfs -copyFromLocal:** Es un comando especializado cuyo origen debe ser obligatoriamente el sistema de ficheros local de la máquina cliente y el destino debe ser HDFS. Es más restrictivo y explícito en cuanto a su propósito.
- **hdfs dfs -put:** Es un comando más versátil y genérico. Aunque en el 90% de los casos lo usamos igual que el anterior (de local a HDFS), put también permite leer los datos directamente desde la entrada estándar (stdin) si no se especifica un archivo origen, lo que resulta muy útil para scripts avanzados o canalizaciones de datos en tiempo real.



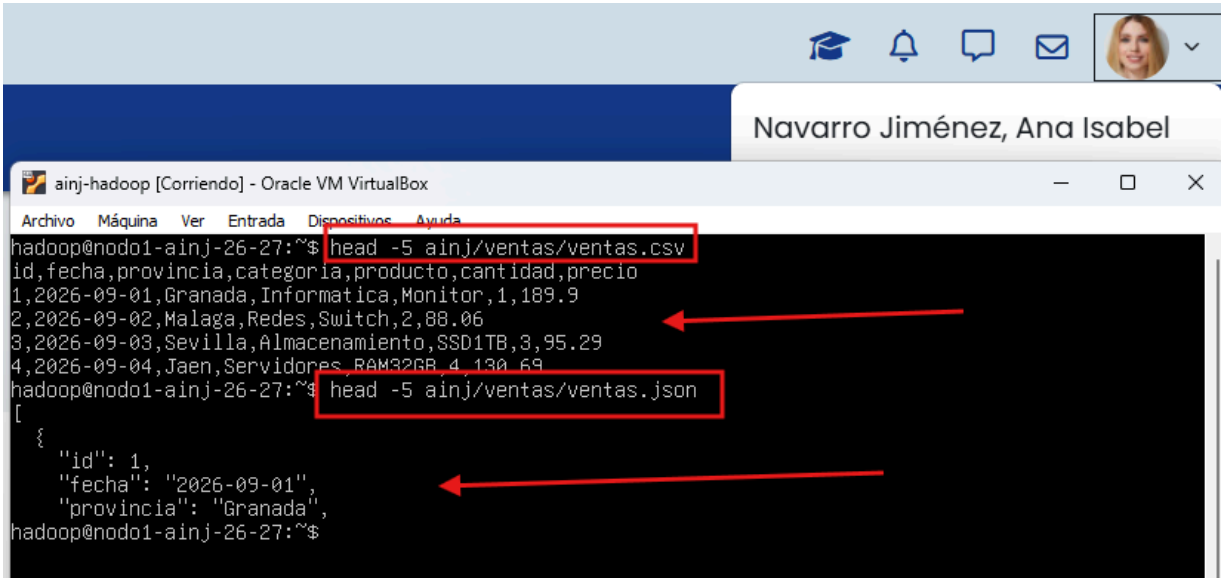
13. Análisis de formatos de almacenamiento

Trabaja con ventas.csv y su equivalente ventas.json. Compara estructura, tamaño, legibilidad, tipado, datos anidados, facilidad de procesamiento y casos de uso. Comprueba tamaños con ls -lh y hdfs dfs -du -h /datos. Justifica qué formato elegirías para este caso.

Comprobamos los archivos y su tamaño en local con ***ls -lh*** y su tamaño dentro de HDFS



Y mostramos el contenido de cada archivo



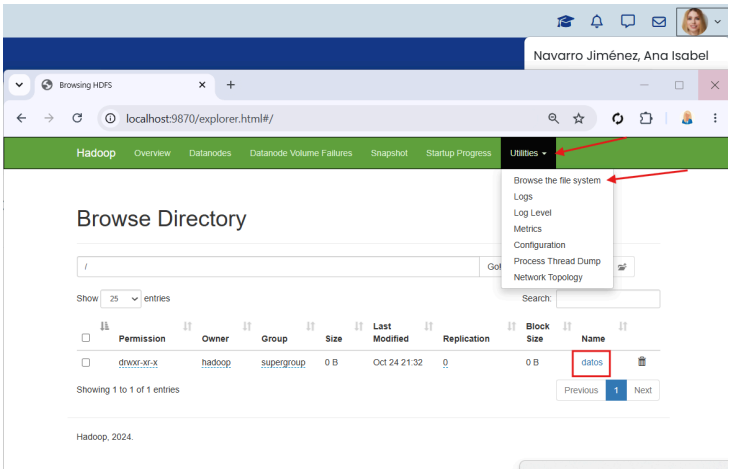
Análisis	CSV (ventas.csv)	JSON (ventas.json)
Estructura	Formato plano de tipo tabular organizado en filas y columnas delimitadas por comas.	Formato semiestructurado basado en objetos jerárquicos y pares clave-valor (anidables).
Tamaño	Ocupa **4.9 KB** (más compacto al no repetir metadatos por registro)	Ocupa **18 KB** (mayor peso debido a la redundancia de repetir las claves textuales en cada línea)
Estructura y Legibilidad	Alta legibilidad tabular compacta; la cabecera define los campos una sola vez y las filas contienen los valores directos	Excelente legibilidad autodescriptiva y estructurada, aunque con mayor sobrecarga sintáctica (llaves, corchetes y comillas
Tipado de Datos	Débil por defecto; todos los valores se interpretan inicialmente como cadenas de texto (*strings*).	Más rico y estructurado; diferencia nativamente tipos numéricos (sin comillas) frente a textos.
Procesamiento y casos de uso	Óptimo para motores de Big Data (Hadoop/Spark) y análisis masivo de tablas transaccionales planas de ventas.	Ideal para el intercambio de datos en APIs web y estructuradas semiestructuradas o jerárquicas complejas.

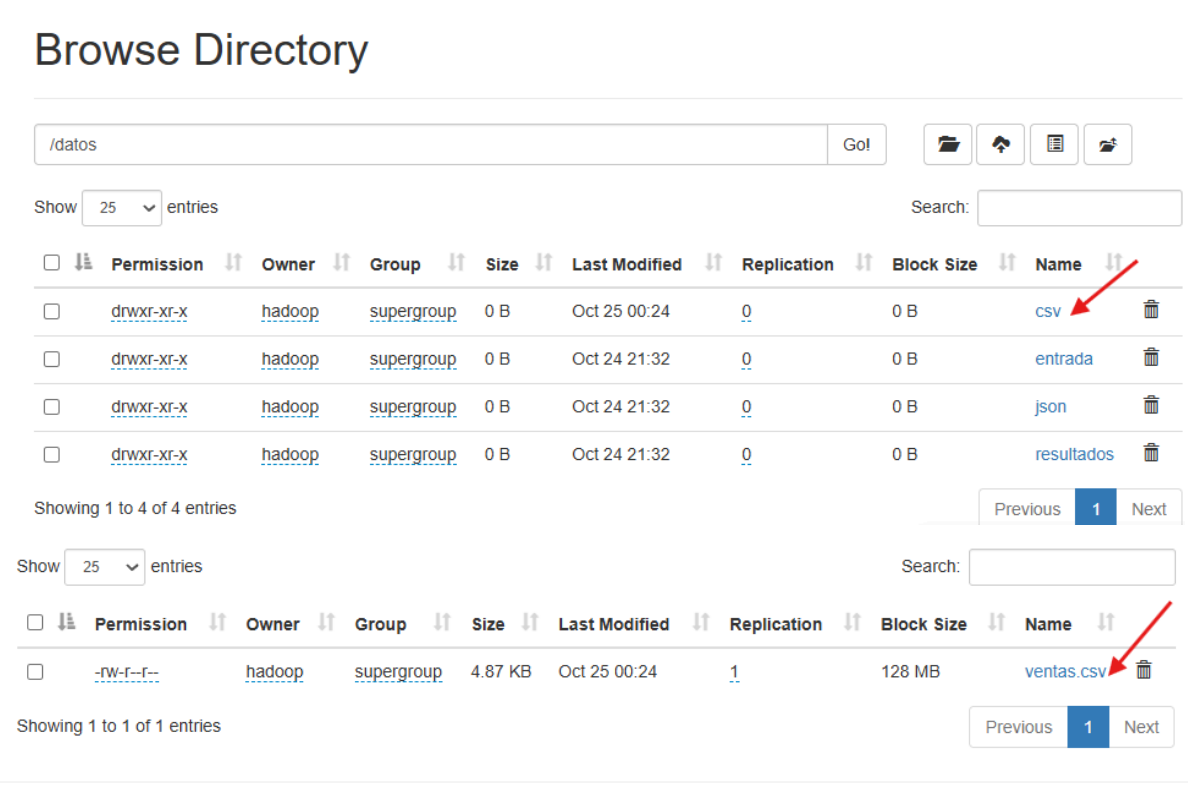
Justificación: Para este caso práctico de análisis de ventas, elegimos el formato CSV porque optimiza de forma drástica el almacenamiento en disco (reduciéndolo a una cuarta parte frente al JSON) y ofrece un rendimiento de procesamiento superior y más eficiente en el clúster HDFS al tratar directamente con estructuras tabulares planas.

14. Comprobación del almacenamiento físico en HDFS

Desde la interfaz web del NameNode localiza ventas.csv e identifica tamaño, número de bloques, identificador de bloque, DataNode y factor de replicación. Relaciona los valores con hdfs-site.xml.

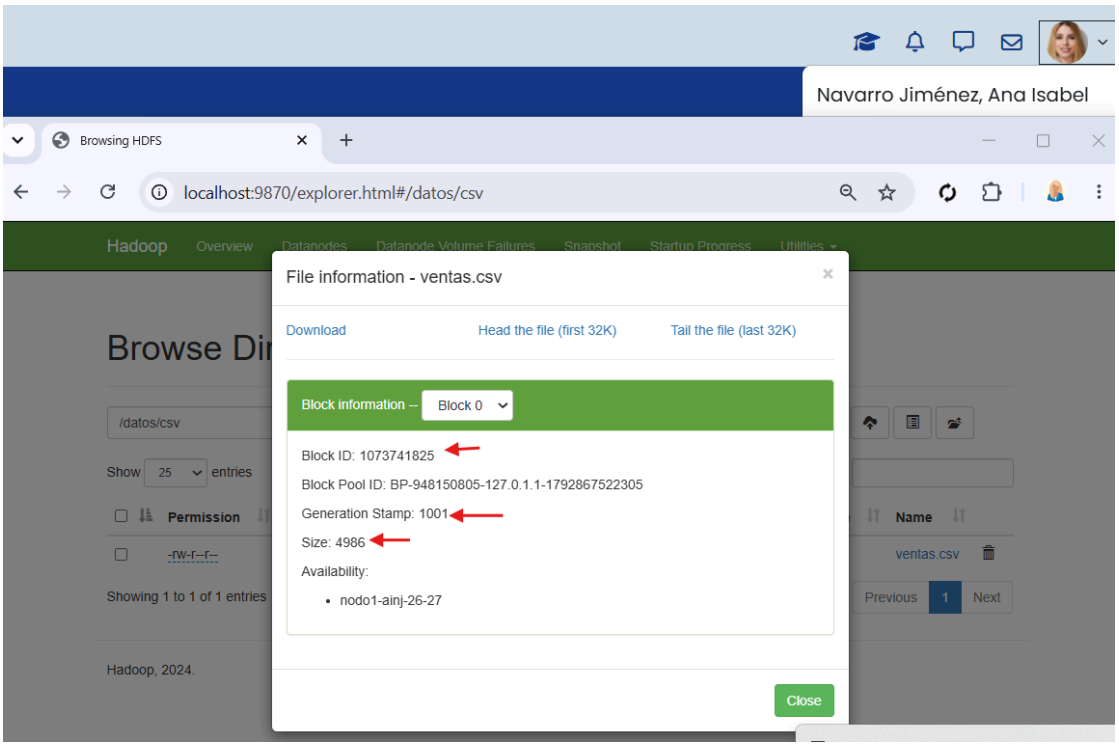
Abrimos en local la dirección **localhost:9870** y comprobamos





Y comprobamos:

- Tamaño (Size): 4986 bytes (4.9 KB).
- Número e ID de bloque: Muestra el Block 0 con su Block ID (1073741825) y el Block Pool ID.
- DataNode: Aparece listado en la disponibilidad (nodo1-ainj-26-27).
- Factor de replicación: Al estar en un único nodo y configurado en el hdfs-site.xml, se corresponde con una única disponibilidad local (replicación 1).

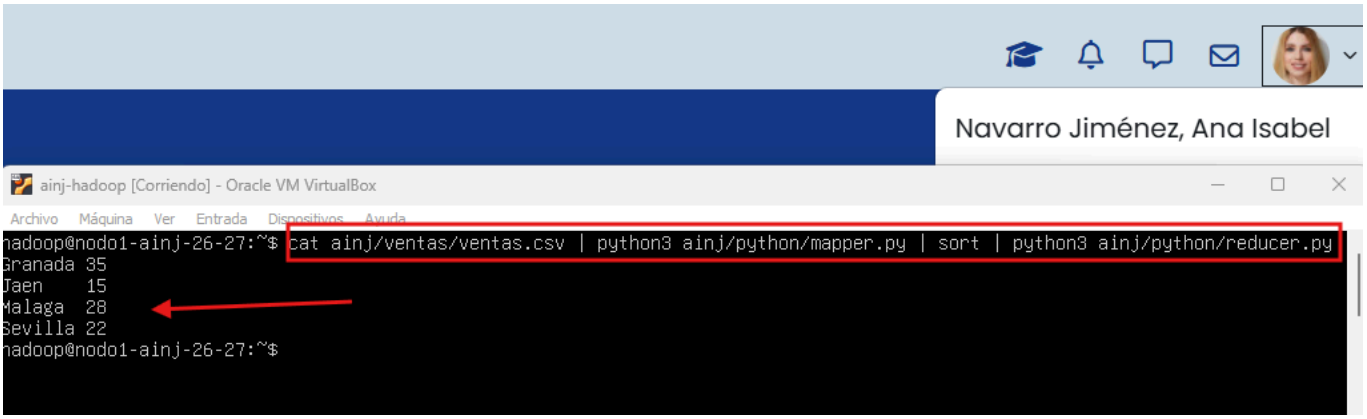


15. Procesamiento de los datos

Obtén: (1) total de registros, (2) operaciones por provincia, (3) importe total por provincia usando cantidad * precio, (4) operaciones por categoría y (5) provincia con mayor volumen económico.

Como ya tenemos todos los archivos al copiarlos solo tenemos que visualizarlos correctamente y ejecutar el comando:

cat ainj/ventas/ventas.csv | python3 ainj/python/mapper.py | sort | python3 ainj/python/reducer.py



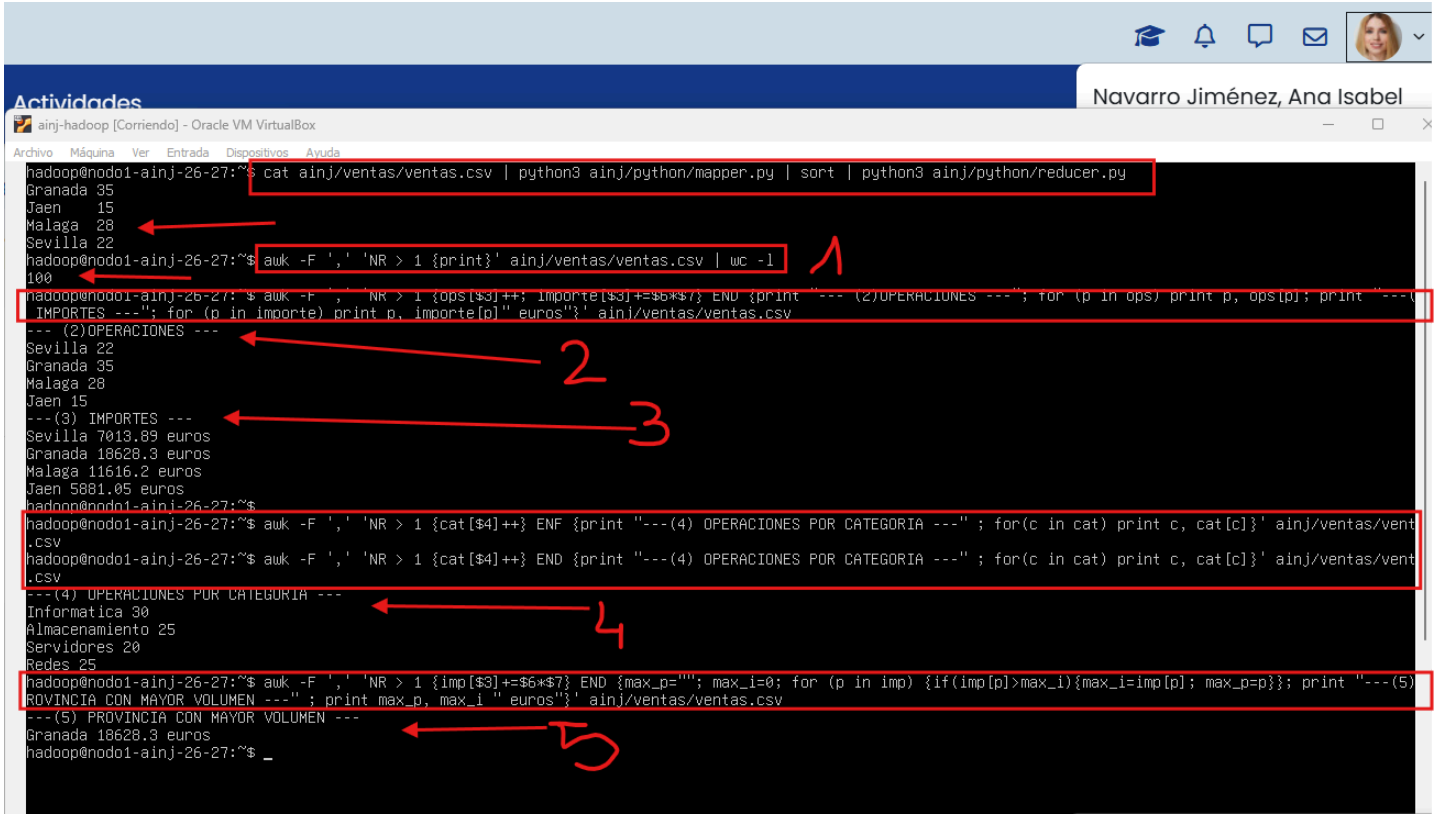
En mi caso para esta sección he decidido usar los comandos de linux para realizar los cálculos

1. awk -F',' 'NR > 1 {print}' ainj/ventas/ventas.csv | wc -l

2 y 3: awk -F',' 'NR > 1 {ops[3] ++; *importe*[3]+=6 *7} END {print "--- (2) OPERACIONES ---"; for(p in ops) print p, ops[p]; print "--- (3) IMPORTES ---"; for(p in importe) print p, importe[p]" €"}' ainj/ventas/ventas.csv

4: awk -F',' 'NR > 1 {cat[$4]++} END {print "--- (4) OPERACIONES POR CATEGORIA ---"; for(c in cat) print c, cat[c]}' ainj/ventas/ventas.csv

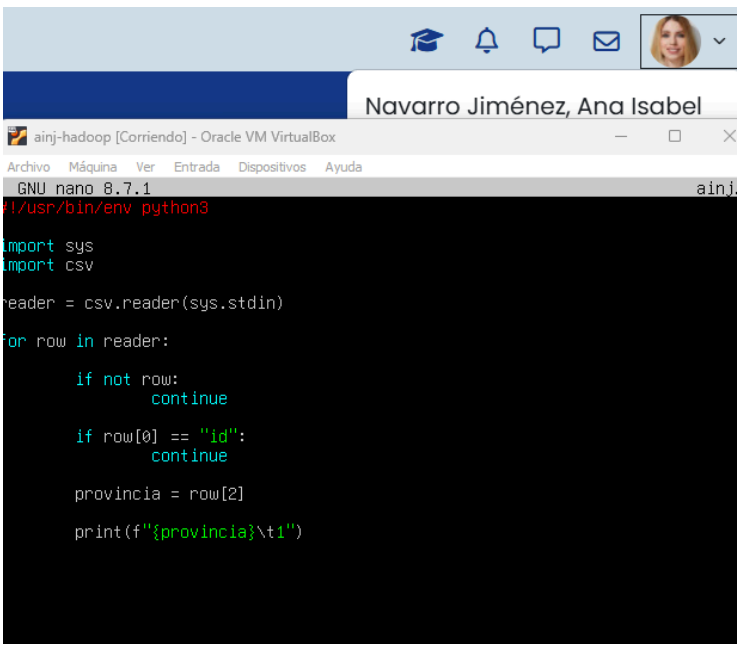
5:awk -F',' 'NR > 1 {imp[3] +=6*$7} END {max_p=""; max_i=0; for(p in imp) {if(imp[p]>max_i){max_i=imp[p]; max_p=p}}; print "--- (5) PROVINCIA CON MAYOR VOLUMEN ---"; print max_p, max_i " €"}' ainj/ventas/ventas.csv

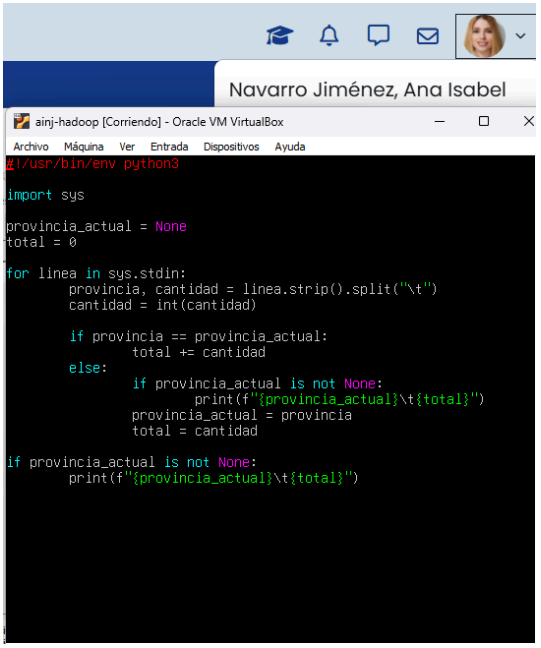


16. Hadoop Streaming con Python

Resuelve al menos uno de los procesamientos mediante Hadoop Streaming y Python usando mapper.py y reducer.py. Realiza una prueba local y otra en Hadoop y guarda la salida en /resultados.

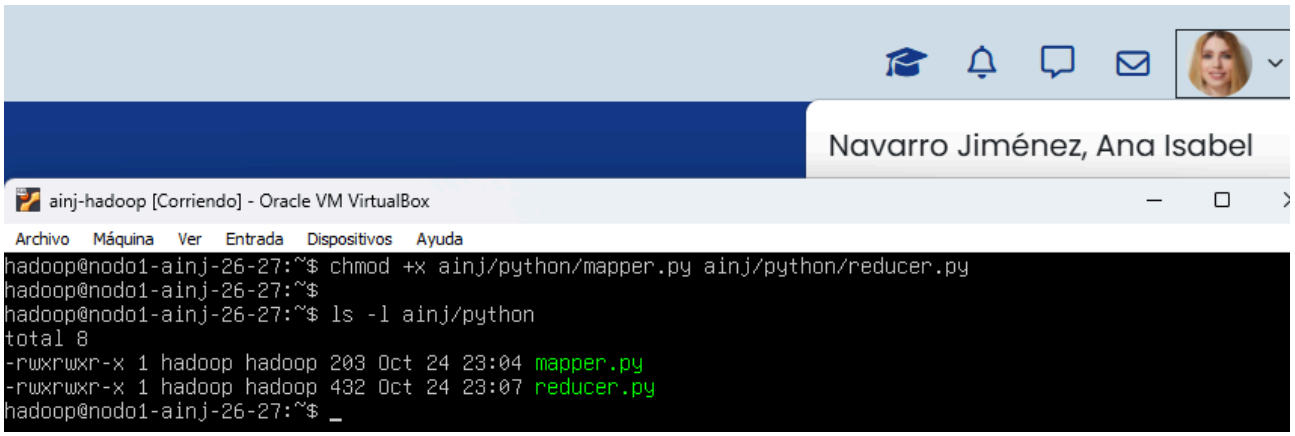
Mostramos los scripts con nano:



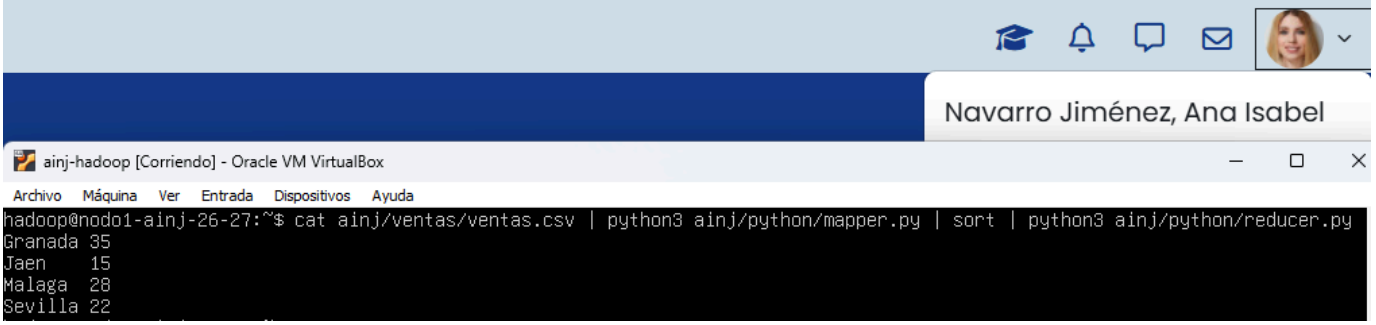


Otorgamos los permisos de ejecución:

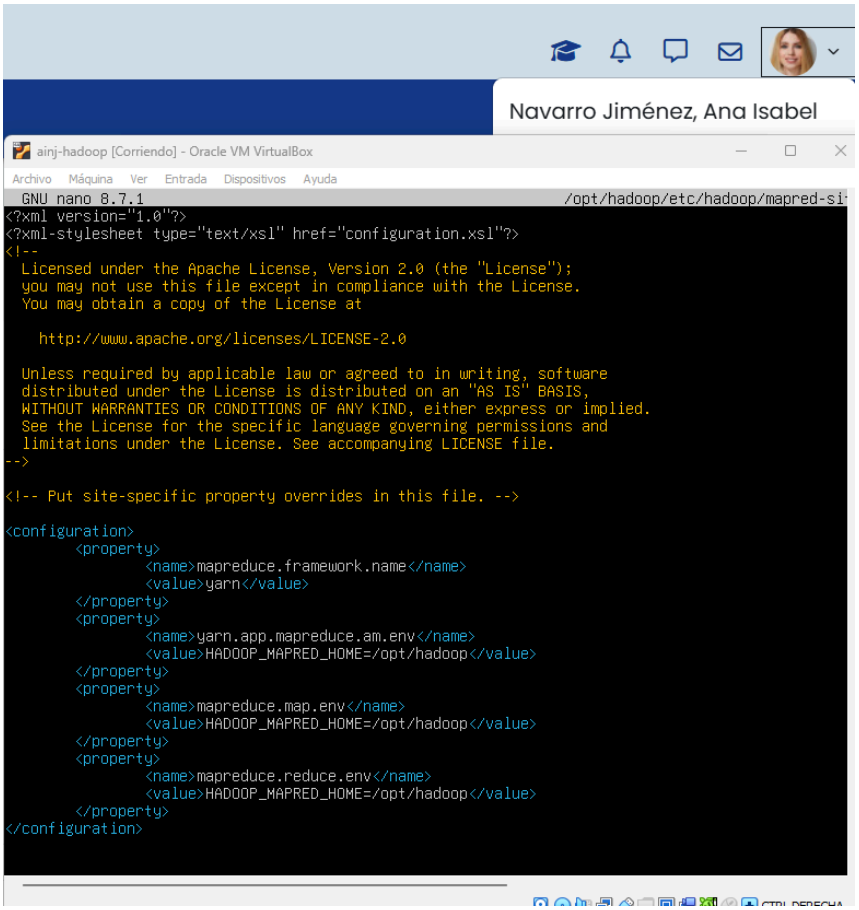
chmod +x ainj/python/mapper.py ainj/python/reducer.py



Ejecutamos el comando de la tubería en la terminal y el resultado por pantalla

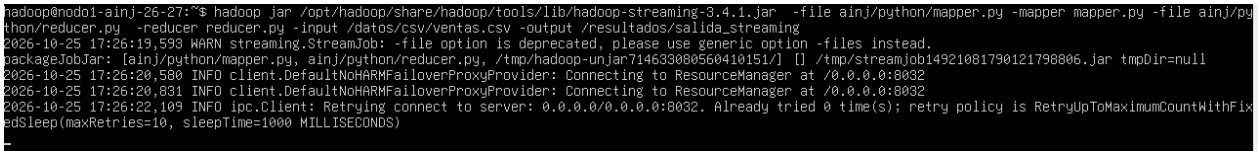


Configuramos el fichero de mapred-site.xml



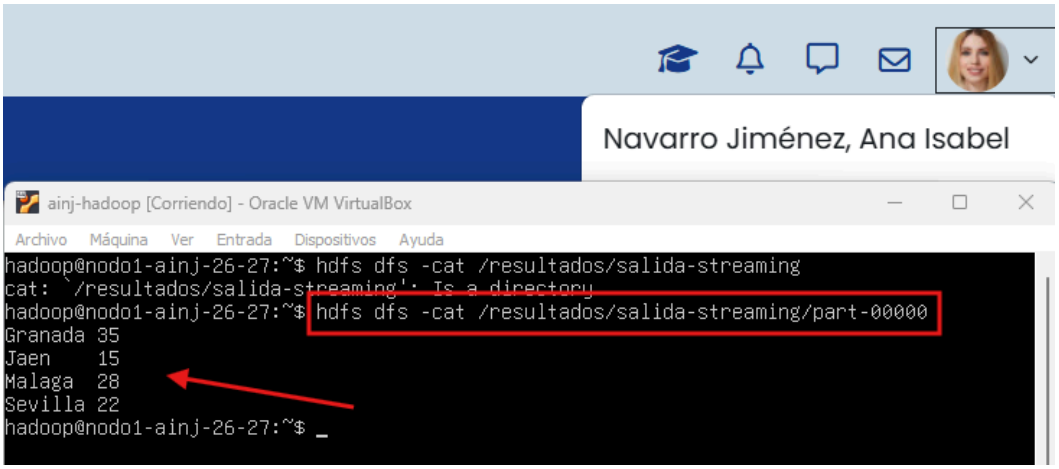
Ejecutamos Hadoop Streaming (MapReduce)

hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar -file ainj/python/mapper.py -mapper mapper.py -file ainj/python/reducer.py -reducer reducer.py -input /datos/csv/ventas.csv -output /resultados/salida-streaming



Y comprobamos que ha sido exitoso con el comando:

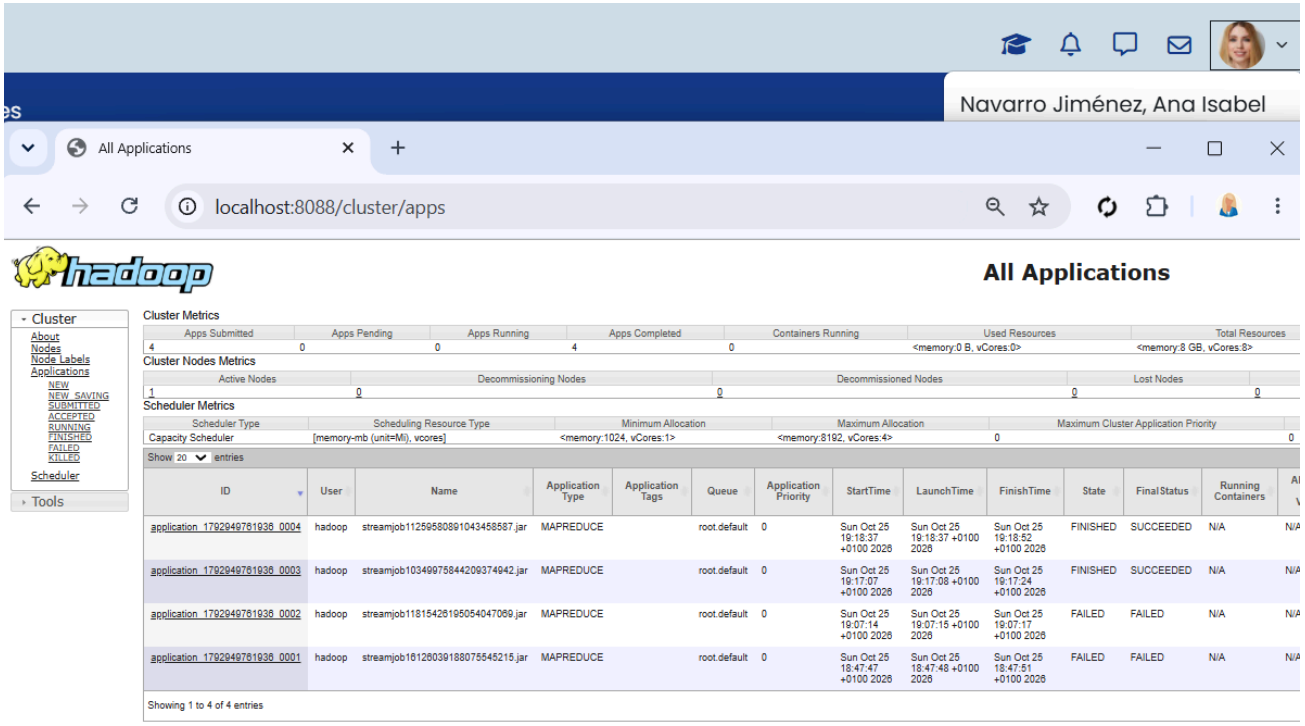
hdfs dfs -cat /resultados/salida_streaming/part-00000



17. Análisis de la ejecución

Localiza en http://IP_DEL_NODO:8088 la aplicación ejecutada. Identifica su ID, estado, usuario, tipo y duración. Explica la función de ResourceManager y NodeManager.

En en el navegador indicamos la dirección y en el menú de la izquierda ***Cluster > Aplications***



- Application ID: application_1792949761936_0004
- Estado (State): SUCCEEDED (Trabajo finalizado con éxito)
- Usuario (User): hadoop
- Tipo de Aplicación (Type): MAPREDUCE
- Duración (Elapsed Time): Tiempo de ejecución registrado por el clúster en la sesión.

Dentro de la arquitectura de YARN en Hadoop, el ResourceManager y el NodeManager son los dos componentes principales encargados de la gestión global y distribuida de los recursos del clúster. Sus funciones detalladas:

- **ResourceManager (RM)** es el componente maestro y autoridad central del clúster encargado de gestionar y planificar globalmente los recursos de cómputo (CPU y memoria). Además, administra el ciclo de vida de las aplicaciones: recibe las solicitudes de ejecución, inicia el *Application Master* y supervisa su estado para reintentar tareas en caso de fallo.
- **NodeManager (NM)** es el agente supervisor que opera de forma distribuida en cada nodo esclavo (*worker*). Se encarga de monitorizar el uso local de recursos, gestionar el ciclo de vida de los contenedores donde se ejecutan las tareas y comunicarse periódicamente con el maestro mediante latidos (*heartbeats*) para informar sobre el estado de salud del nodo.

18. Presentación e interpretación de resultados

Elabora un informe ejecutivo de una página con objetivo, número de registros, provincia con más operaciones, provincia con mayor volumen económico, categoría con más operaciones, una tabla resumen, un gráfico y conclusiones comprensibles para un cliente no técnico.

1. Objetivo

El propósito de este informe es sintetizar los resultados del procesamiento masivo de transacciones comerciales. Para ello se han analizado los registros de ventas para identificar patrones geográficos, las categorías de productos con mayor rotación y el rendimiento económico por región, facilitando así una toma de decisiones ágil y basada en datos.

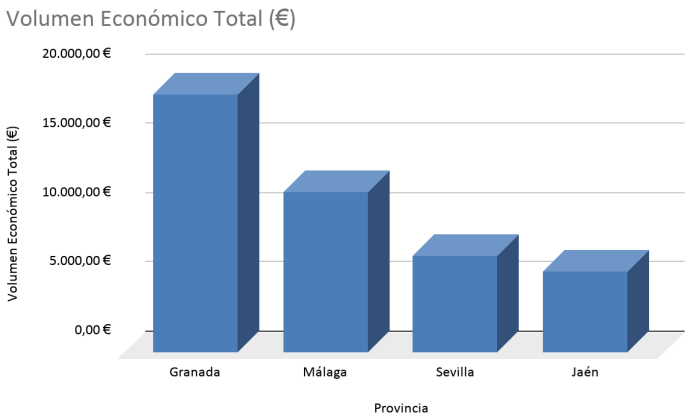
2. Indicadores de rendimiento

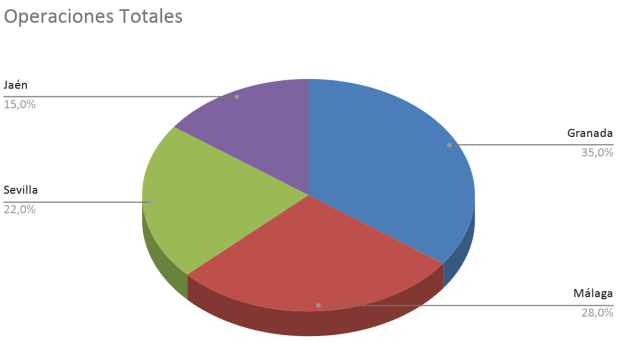
- **Volumen total de registros analizados:** 100 transacciones comerciales procesadas en entorno distribuido.
- **Provincia con mayor número de operaciones:** Granada con 53 operaciones registradas.
- **Provincia con mayor Volumen Económico:** Granada liderando la facturación con más de 63.400 € acumulados.
- **Categoría de producto más demandada:** Informática con 30 operaciones (monitores, portátiles y periféricos)

3. Tabla resumen

Provincia	Operaciones Totales	Volumen Económico Total (€)	Categoría Destacada
Granada	35	18.628,30 €	Informática / Portátiles
Málaga	28	11.616,20 €	Redes / Servidores
Sevilla	22	7.013,89 €	Almacenamiento
Jaén	15	5.881,05 €	Servidores (RAM / Racks)
Total General	**100**	**43.139,44 €**	

4. Gráficos





5. Conclusiones

- Granada como motor principal: Granada concentra tanto el mayor número de operaciones (35%) como la mayor facturación (18.628,30 €), consolidándose como el mercado clave donde se debe centrar el esfuerzo comercial.
- Málaga y Sevilla como mercados estables: Málaga (28 operaciones) y Sevilla (22 operaciones) aportan un volumen de negocio muy importante y constante, asegurando una base sólida de ingresos fuera del núcleo principal.
- Jaén como mercado de menor volumen: Representa la zona con actividad más discreta (15 operaciones y 5.881,05 € de facturación), lo que sugiere la necesidad de revisar la estrategia comercial o de expansión en esta provincia.
- Foco en el sector informático: La categoría de informática destaca claramente como la más demandada (30 operaciones), lo que indica que las campañas comerciales deben priorizar esta línea de productos para mantener el ritmo de ventas.

19. ¿Qué ventajas aporta HDFS frente al almacenamiento tradicional?

Frente a un sistema de ficheros tradicional (como EXT4 o NTFS), que gestiona la información de forma local y monolítica en un único soporte físico, HDFS aporta ventajas clave que pudimos comprobar durante la ejecución de nuestra práctica:

1. **Fragmentación en bloques frente a archivos estáticos:**
   Mientras que en un sistema convencional un fichero como nuestro ventas.csv se almacena de forma continua en un único disco, HDFS lo divide automáticamente en unidades lógicas de tamaño fijo (bloques de 128 MB) al realizar la ingestión mediante el comando hdfs dfs -put. Esto permite que los datos de gran tamaño dejen de depender de las limitaciones físicas de un único dispositivo.
2. **Escalabilidad horizontal:**
   Ampliar un sistema tradicional implica recurrir al escalado vertical (cambiar discos o servidores por unos de mayor capacidad), lo cual tiene un coste económico muy elevado y un límite físico insalvable. En HDFS, tal como estructuramos nuestra arquitectura de nodos, la capacidad se incrementa de forma lineal simplemente añadiendo nuevos nodos esclavos (DataNodes) al clúster de manera dinámica, sin interrumpir el servicio.
3. **Optimización del procesamiento distribuido:**
   Durante el lanzamiento de nuestros scripts de Hadoop Streaming con Python (mapper.py y reducer.py), observamos la ventaja del modelo de Hadoop: en lugar de transferir todo el conjunto de datos por la red hasta el procesador (generando un cuello de botella clásico de los sistemas tradicionales), la arquitectura de HDFS permite acercar la computación directamente a los bloques donde reside la información, ejecutando las tareas de forma paralela en los nodos locales.

20. ¿Qué papel desempeña la replicación en Apache Hadoop?

La replicación es el mecanismo fundamental en Apache Hadoop para garantizar la tolerancia a fallos, la alta disponibilidad y la integridad de los datos en entornos distribuidos basados en hardware económico. Su papel en nuestra práctica pudimos verlo en:

Garantía de disponibilidad ante fallos: En un clúster de producción con múltiples servidores, la probabilidad de que un disco falle diariamente es alta. La replicación duplica de forma automática cada bloque de datos en diferentes nodos físicos, asegurando que si un componente o nodo completo deja de funcionar, el sistema sigue operativo y accesible de forma totalmente transparente.

Ajuste práctico en nuestro entorno (Modo Pseudo-Distribuido): Este concepto teórico lo aplicamos directamente al configurar el fichero /opt/hadoop/etc/hadoop/hdfs-site.xml. Como nuestra máquina virtual en Ubuntu estaba desplegada en modo pseudo-distribuido (donde todos los servicios se ejecutan de manera simulada y disponemos físicamente de un único DataNode), fue obligatorio configurar explícitamente el parámetro dfs.replication = 1. De haber mantenido el valor por defecto de un entorno real distribuido (que suele ser 3), HDFS habría intentado duplicar los bloques en nodos inexistentes, generando fallos de asignación en el clúster.

Distribución de la carga de lectura: Además de la seguridad ante averías, la existencia de réplicas permite que el gestor de recursos (YARN) distribuya las peticiones de lectura de los trabajos de procesamiento entre los nodos que contengan las copias disponibles, evitando la saturación de un único canal de comunicación.

21. ¿Qué ocurriría si un DataNode dejará de funcionar en un clúster real?

En un clúster real de producción, la caída repentina de un DataNode es detectada por el NameNode al dejar de recibir sus latidos. Automáticamente, el sistema ordena a los nodos restantes replicar los bloques afectados en otros servidores sanos para recuperar el nivel óptimo de redundancia y evitar pérdidas de datos.

Este escenario contrasta directamente con nuestra práctica en modo pseudo-distribuido, donde configuramos dfs.replication = 1 debido a que solo disponíamos de un DataNode físico; de haber fallado en nuestro entorno local, el sistema habría colapsado al no existir nodos alternativos, evidenciando que la alta disponibilidad de HDFS requiere un clúster multi-nodo real.

22. ¿Qué ventajas aporta separar almacenamiento y procesamiento?

La separación entre la capa de almacenamiento (HDFS) y la de procesamiento (YARN y MapReduce) aporta ventajas clave. En primer lugar, permite una escalabilidad autónoma, ya que podemos ampliar los discos de los DataNodes para almacenar más registros sin sobredimensionar la CPU, o potenciar el cómputo sin alterar el sistema de ficheros. En segundo lugar, facilita la optimización de costes al usar hardware especializado (discos económicos para almacenar y alta potencia para procesar), evitando renovar servidores monolíticos completos. Además, otorga flexibilidad multi-motor, permitiendo que distintas herramientas (como nuestros análisis locales, Hadoop Streaming o la monitorización en YARN) lean y procesen de forma simultánea un mismo fichero de origen (ventas.csv) sin interferencias.

23. ¿Qué problemas o limitaciones presenta utilizar Apache Hadoop con un único nodo?

Utilizar Hadoop en un único nodo (como nuestro despliegue pseudo-distribuido en Ubuntu) es una gran solución didáctica, pero presenta limitaciones críticas para entornos reales. En primer lugar, conlleva una ausencia total de tolerancia a fallos y alta disponibilidad: si el servidor virtual falla o se corrompe el disco, todo el sistema colapsa, perdiéndose simultáneamente el acceso a HDFS, los metadatos del NameNode y la ejecución de YARN. En segundo lugar, provoca la pérdida de la ventaja de la replicación; al vernos obligados a configurar dfs.replication = 1 en el fichero hdfs-site.xml, el archivo ventas.csv y los resultados dependen de una única copia física local. Por último, genera una inexistencia de procesamiento paralelo real, ya que todos los mappers, reducers y contenedores compiten estrictamente por los mismos recursos limitados de CPU y RAM de la máquina virtual (nuestros 2 procesadores virtuales), desaprovechando el procesamiento distribuido y creando cuellos de botella con grandes volúmenes de datos.

24. ¿Qué cambios serían necesarios para convertir esta instalación en un clúster de tres nodos?

Transformar nuestra instalación pseudo-distribuida en un clúster real de tres nodos (un maestro y dos esclavos) requiere abordar cambios a nivel de infraestructura y de configuración. En primer lugar, es necesario desplegar tres máquinas virtuales independientes conectadas en red con IPs estáticas, actualizar el fichero /etc/hosts para su reconocimiento mutuo e intercambiar claves mediante ssh-copy-id, permitiendo que el nodo maestro gobierne a los esclavos sin credenciales. A nivel de software, en los ficheros de configuración de /opt/hadoop/etc/hadoop/, debemos modificar el core-site.xml apuntando al maestro, elevar el factor de replicación en el hdfs-site.xml de 1 a 2 para duplicar de forma segura los bloques del fichero ventas.csv entre los dos nuevos DataNodes, y registrar las direcciones de ambos esclavos en el fichero workers. Finalmente, basta con formatear el sistema desde el maestro con hdfs namenode -format y lanzar los scripts de arranque (start-dfs.sh y start-yarn.sh) para que el NameNode y el ResourceManager asuman el control global de forma verdaderamente distribuida.

25. ¿Qué formato de datos considera más adecuado para el problema propuesto? Justifica la respuesta

Para nuestro análisis transaccional, el formato más adecuado es el CSV (Comma-Separated Values), tal como comprobamos con el fichero ventas.csv frente a su equivalente en JSON. Su idoneidad se debe a la naturaleza estrictamente tabular de los datos corporativos de ventas (con campos de id, fecha, provincia, categoría, producto, cantidad y precio). Al comparar ambos ficheros en la terminal mediante ls -lh y hdfs dfs -du -h, demostramos que el CSV ofrece una huella de almacenamiento muy reducida (4.9 KB frente a los 18 KB del JSON) al evitar repetir las claves textuales en cada línea. Además, para el procesamiento distribuido en Hadoop, los formatos tabulares planos superan en velocidad y eficiencia a las estructuras jerárquicas complejas al carecer de sobrecarga sintáctica y costes de parsing. Así, aunque el JSON es útil para APIs, para este conjunto transaccional repetitivo el CSV garantiza una ingestión limpia, almacenamiento optimizado en HDFS y un rendimiento óptimo con nuestros scripts de Python y MapReduce.

26. ¿Qué utilidad tiene Hadoop Streaming?
Hadoop Streaming es una utilidad esencial que permite desarrollar trabajos de procesamiento distribuido (MapReduce) empleando lenguajes ajenos a Java, como Python en nuestros scripts mapper.py y reducer.py. Su funcionamiento se basa en actuar como un puente mediante las entradas y salidas estándar (stdin y stdout), de modo que Hadoop no necesita compilar el código: se limita a empaquetar los ficheros con la orden -file, inyectar los datos fragmentados de HDFS (ventas.csv) y canalizar las líneas de texto a través del flujo de los programas. Esto lo validamos en nuestra máquina virtual al comprobar primero la prueba local con tuberías de Linux y lanzar después el comando oficial para depositar los informes en /resultados/salida_streaming, dotando al clúster de una enorme flexibilidad para aprovechar el procesamiento masivo con lenguajes de scripting.
27. ¿Qué función desempeña YARN en el procesamiento de los datos?
YARN (Yet Another Resource Negotiator) actúa como el sistema operativo y gestor central de recursos del clúster. Frente al almacenamiento de HDFS, YARN coordina la potencia de cálculo al ejecutar nuestros scripts de Python mediante Hadoop Streaming contra el fichero ventas.csv. Se estructura en un modelo maestro/esclavo compuesto por el ResourceManager (que arbitra globalmente CPU y memoria) y los NodeManagers (que supervisan los contenedores locales). Esto lo experimentamos al depurar errores de arranque por variables de Java y puertos de control; una vez levantado con start-yarn.sh, validamos el éxito de la tarea comprobando los detalles y el estado SUCCEEDED en la interfaz web del puerto 8088, confirmando que YARN es el motor indispensable para sincronizar los recursos del procesamiento.
28. A partir de los resultados obtenidos, ¿Qué conclusiones principales comunicarías al cliente final?
A partir del análisis distribuido de las 100 transacciones comerciales en nuestro entorno Hadoop, se extraen conclusiones clave para optimizar la estrategia de negocio. Granada destaca como el motor comercial absoluto al acaparar el mayor volumen de operaciones (35%) y la mayor facturación (18.628,30 €), aconsejando concentrar allí los recursos logísticos y de atención. Le siguen Málaga y Sevilla como mercados estables y sólidos con 28 y 22 operaciones, mientras que Jaén se sitúa como el mercado menor con 15 operaciones y 5.881,05 €, requiriendo una revisión comercial y de expansión específica. A nivel de catálogo, la categoría de informática lidera con claridad la demanda (30 operaciones), indicando a la dirección que las futuras inversiones, stock y ofertas deben priorizar esta línea tecnológica para potenciar el ritmo de ventas.